

Dispensa di Machine Learning

Corso 654AA — Laurea Magistrale in Informatica (Curriculum AI)

Università di Pisa — Prof. Alessio Micheli

Anno Accademico 2025/2026

“The problem of learning is arguably at the very core of the problem of intelligence, both biological and artificial.”

— Poggio & Shelton, AI Magazine 1999

Contents

1	Introduzione al Machine Learning	9
1.1	Cos'è il Machine Learning?	9
1.1.1	Quando usare il Machine Learning?	9
1.1.2	ML vs. Programmazione Tradizionale	10
1.2	Il Framework di Apprendimento come Approssimazione di Funzione	10
1.2.1	I Cinque Elementi Fondamentali	10
1.2.2	Notazione sui Dati	11
1.3	Tipologie di Task	11
1.3.1	Supervised Learning	11
1.3.2	Unsupervised Learning	11
1.3.3	Altri tipi di apprendimento	11
1.4	Generalizzazione, Overfitting e Inductive Bias	12
1.4.1	Il Problema della Generalizzazione	12
1.4.2	Overfitting	12
1.4.3	Trade-off Bias-Variance	12
1.4.4	Inductive Bias	13
1.5	Hypothesis Space e Parametri	13
1.6	Struttura del Corso	14
2	Modelli Lineari	15
2.1	Introduzione ai Modelli Lineari	15
2.2	Linear Threshold Unit (LTU) e Perceptron	15
2.2.1	Il Modello	15
2.2.2	Confine di Decisione	16
2.2.3	Algoritmo del Perceptron	16
2.3	Regressione Lineare e Least Mean Squares (LMS)	17
2.3.1	Modello per Regressione	17
2.3.2	Funzione di Errore (Loss)	17
2.3.3	Soluzione Analitica (Least Squares)	17
2.3.4	Algoritmo LMS (Widrow-Hoff / Delta Rule)	17
2.4	Gradient Descent: Online, Batch e Mini-batch	18

2.5	Classificazione Lineare	18
2.5.1	Classificazione Binaria	18
2.5.2	Multiclasse (One-vs-All)	19
2.6	Limitazioni dei Modelli Lineari e Funzioni di Attivazione Non Lineari .	19
3	K-Nearest Neighbors (K-nn)	20
3.1	Introduzione: Approcci Lazy vs. Eager	20
3.2	1-Nearest Neighbor (1-nn)	20
3.2.1	Proprietà del 1-nn	21
3.2.2	Diagramma di Voronoi	21
3.3	K-Nearest Neighbors	21
3.3.1	Generalizzazione a K Vicini	21
3.3.2	Effetto di K	21
3.4	Misure di Distanza	22
3.5	Analisi Teorica: Curse of Dimensionality	22
3.6	Proprietà e Limiti Pratici	22
3.7	Confronto: Modelli Locali vs. Globali	23
4	Reti Neurali Artificiali — Parte 1	24
4.1	Motivazione e Obiettivi	24
4.2	Il Neurone Biologico e il Modello Artificiale	24
4.2.1	Neurone Biologico	24
4.2.2	Neurone Artificiale (Processing Unit)	25
4.3	Funzioni di Attivazione	25
4.3.1	Funzione a Gradino (Step/Heaviside)	25
4.3.2	Funzione Lineare (Identità)	25
4.3.3	Sigmoide Logistica	25
4.3.4	Tangente Iperbolica (tanh)	26
4.3.5	ReLU (Rectified Linear Unit)	26
4.3.6	Softmax (per output layer multiclasse)	26
4.4	Multi-Layer Perceptron (MLP)	26
4.4.1	Architettura	26
4.4.2	Forward Pass	27
4.4.3	Teorema di Approssimazione Universale	27
4.5	Funzione Obiettivo e Task	27
4.5.1	Regressione: Sum-of-Squares Error	27
4.5.2	Classificazione: Cross-Entropy Loss	28
4.6	Rappresentazione: Cosa Può Imparare un MLP?	28
4.7	Gradient Descent per MLP: Introduzione alla Backpropagation	28

5	Reti Neurali Artificiali — Parte 2: Backpropagation e Training	30
5.1	L'Algoritmo di Backpropagation	30
5.1.1	Derivazione per l'Output Layer	30
5.1.2	Derivazione per i Layer Nascosti	31
5.1.3	Algoritmo Completo	31
5.1.4	Derivata delle Funzioni di Attivazione	31
5.2	Euristiche e Iperparametri per il Training	32
5.2.1	Inizializzazione dei Pesi	32
5.2.2	Learning Rate η	32
5.2.3	Online vs. Batch vs. Mini-batch	32
5.2.4	Momentum	33
5.2.5	Criteri di Stop	33
5.3	Minimi Locali e Non-Convessità	33
5.4	Overfitting e Regularizzazione	33
5.4.1	Weight Decay (Regularizzazione L2)	33
5.4.2	Early Stopping	34
5.4.3	Numero di Hidden Units	34
5.4.4	Dropout (Cenni)	34
5.5	Rappresentazione dell'Input e dell'Output	34
5.5.1	Normalizzazione dell'Input	34
5.5.2	Codifica dell'Output	35
5.6	Cascade Correlation (Cenni)	35
6	Model Assessment e Model Selection	36
6.1	Introduzione: Il Problema della Valutazione	36
6.2	Definizioni Formali di Errore	36
6.2.1	Training Error (Errore Empirico)	36
6.2.2	Generalization Error (Rischio)	37
6.2.3	Test Error	37
6.3	Hold-Out e Splitting dei Dati	37
6.3.1	Schema a Tre Insiemi	37
6.4	Cross-Validation	37
6.4.1	K-Fold Cross-Validation	38
6.4.2	Uso Corretto della Cross-Validation	38
6.4.3	Schema Nested Cross-Validation	38
6.5	Bias e Varianza della Stima dell'Errore	39
6.6	Bootstrap	39
6.7	Metriche di Valutazione	40
6.7.1	Classificazione	40

6.7.2	Regressione	40
6.8	Selezione del Modello: AIC e BIC	40
7	Statistical Learning Theory e VC Dimension	41
7.1	Motivazione: Basi Teoriche della Generalizzazione	41
7.2	Setup Formale	41
7.3	VC Dimension	41
7.3.1	Esempi di VC Dimension	42
7.4	Structural Risk Minimization (SRM)	42
7.5	Sample Complexity	43
7.6	Criteri Analitici per la Selezione del Modello	43
7.7	Limiti Pratici della SLT	43
8	Support Vector Machines (SVM)	45
8.1	Introduzione e Motivazione	45
8.2	SVM a Margine Massimo (Hard Margin)	45
8.2.1	Setup	45
8.2.2	Il Concetto di Margine	45
8.2.3	Problema di Ottimizzazione	46
8.2.4	Formulazione Duale	46
8.3	Soft Margin SVM	46
8.3.1	Ruolo di C	47
8.4	Kernel Trick	47
8.4.1	Feature Map non Lineare	47
8.4.2	Kernel Function	47
8.4.3	Kernel Comuni	47
8.5	SVM per Regressione (SVR)	48
8.6	Pro e Contro delle SVM	48
9	Bias-Variance Tradeoff e Ensemble Methods	49
9.1	Decomposizione Bias-Varianza	49
9.1.1	Setup Formale	49
9.1.2	Decomposizione	49
9.1.3	Interpretazione	50
9.1.4	Esempi	50
9.2	Ensemble Methods	50
9.2.1	Effetto dell'Ensemble sulla Varianza	51
9.2.2	Bagging (Bootstrap Aggregating)	51
9.2.3	Random Forest	51
9.2.4	Boosting	51

9.2.5	Gradient Boosting	52
9.2.6	Stacking	52
9.3	Bias-Varianza per Classificazione	52
10	Convolutional Neural Networks (CNN)	53
10.1	Motivazione: Elaborazione di Dati Strutturati	53
10.2	L'Operazione di Convoluzione	53
10.2.1	Convoluzione 1D	53
10.2.2	Convoluzione 2D (per Immagini)	54
10.2.3	Padding e Stride	54
10.3	Architettura CNN	54
10.3.1	Convolutional Layer	54
10.3.2	Pooling Layer	55
10.3.3	Fully-Connected Layer	55
10.3.4	Schema Tipico (LeNet-like)	55
10.4	Proprietà delle CNN	55
10.5	Architetture CNN Importanti	56
10.5.1	Residual Connections (ResNet)	56
10.6	Training delle CNN	56
11	Deep Neural Networks	57
11.1	Cos'è il Deep Learning?	57
11.1.1	Shallow vs. Deep Models	57
11.1.2	Fattori Abilitanti del Deep Learning (dal 2010)	57
11.2	Rappresentazione Gerarchica	58
11.3	Perché i Layer Multipli Sono Utili?	58
11.4	Tecniche per il Training di Reti Profonde	58
11.4.1	Vanishing/Exploding Gradient	58
11.4.2	Batch Normalization (BatchNorm)	58
11.4.3	Dropout	59
11.4.4	Ottimizzatori Adattativi	59
11.4.5	Pre-Training (Layer-wise)	60
11.5	Architetture Deep: Overview	60
11.6	Transfer Learning e Fine-Tuning	60
12	Randomized Neural Networks	61
12.1	Introduzione agli Approcci Randomizzati	61
12.1.1	Randomness in ML: Livelli di Applicazione	61
12.2	Extreme Learning Machine (ELM)	61
12.2.1	Analisi	62

12.3 Random Kitchen Sinks e Random Features	62
12.4 Reservoir Computing ed Echo State Network	62
12.5 Random Forests come Randomized ML	63
12.6 Dropout come Randomizzazione	63
12.7 Vantaggi e Limiti degli Approcci Randomizzati	63
13 Unsupervised Learning con Reti Neurali	65
13.1 Unsupervised Learning: Introduzione	65
13.2 Clustering e Vector Quantization	65
13.2.1 Il Problema del Clustering	65
13.2.2 Vector Quantization (VQ)	66
13.2.3 K-Means	66
13.3 Self-Organizing Map (SOM)	66
13.3.1 Architettura	66
13.3.2 Algoritmo di Apprendimento	67
13.3.3 Proprietà della SOM	67
13.4 Autoencoder	67
13.4.1 Variational Autoencoder (VAE)	68
14 Recurrent Neural Networks (RNN)	69
14.1 Dati Sequenziali e la Necessità di Memoria	69
14.1.1 Approcci per la Memoria nelle NN	69
14.2 Recurrent Neural Network (Simple RNN)	69
14.2.1 Architettura	69
14.2.2 Tipi di Task per RNN	70
14.3 Backpropagation Through Time (BPTT)	70
14.4 Long Short-Term Memory (LSTM)	71
14.4.1 Gated Recurrent Unit (GRU)	72
14.5 Architetture RNN Avanzate	72
14.5.1 Bidirectional RNN	72
14.5.2 Stacked (Multi-layer) RNN	72
14.5.3 Seq2Seq e Attention	72
14.6 RNN vs. Transformers	73
15 Learning in Structured Domains e Graph Neural Networks	74
15.1 Motivazione: Oltre i Dati Piatti	74
15.2 Reti Neurali per Strutture: Cenni Storici	74
15.3 Il Framework Deep Graph Networks (DGN)	75
15.3.1 Rappresentazione del Grafo	75
15.3.2 Il Principio del Neighborhood Aggregation	75

15.4 Graph Convolutional Network (GCN)	76
15.4.1 Graph Neural Network (Scarselli et al., 2009)	76
15.5 Graph Attention Networks (GAT)	76
15.6 Graph-Level Readout	76
15.7 Espressività delle GNN	77
15.8 Reservoir Computing per Grafi: GraphESN	77
15.9 Sfide Aperte nel Deep Learning per Grafi	77
A Riassunto della Notazione	78
B Derivazione Completa della Backpropagation	81

Chapter 1

Introduzione al Machine Learning

1.1 Cos'è il Machine Learning?

Il **Machine Learning** (ML) è un'area dell'Intelligenza Artificiale che studia principi, metodi e algoritmi per l'apprendimento automatico e la predizione a partire dai dati. Il suo obiettivo centrale è permettere a un sistema di *imparare dall'esperienza*: dati esempi noti, il sistema costruisce un *modello capace di generalizzare*, ovvero di produrre risposte accurate su dati mai visti.

Definizione 1.1: Machine Learning

Il Machine Learning consiste nell'inferenza di un modello (o ipotesi) da un insieme di esempi, in modo tale che il modello possa essere usato per fare *predizioni accurate su nuovi dati non osservati*. Formalmente, *si cerca una funzione $h : \mathcal{X} \rightarrow \mathcal{Y}$ che approssimi una funzione target sconosciuta f* .

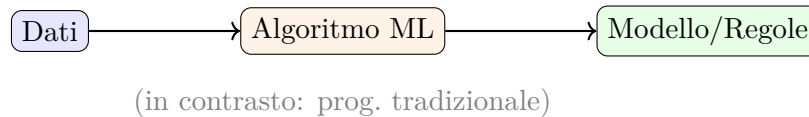
1.1.1 Quando usare il Machine Learning?

Il ML risulta particolarmente utile nelle seguenti situazioni:

- **Assenza di teoria**: non esiste una teoria o conoscenza a priori sufficiente per formalizzare il problema (es. riconoscimento del viso: gli umani lo fanno ma non sanno descrivere *come*).
- **Dati rumorosi o incompleti**: i dati disponibili sono incerti, rumorosi o parziali, rendendo difficile la formalizzazione analitica di una soluzione.
- **Comportamento personalizzato**: il sistema deve adattarsi alle preferenze specifiche dell'utente (es. filtri email, interfacce intelligenti).
- **Conoscenza umana insufficiente**: non si dispone di sufficiente conoscenza a priori (es. predizione dell'affinità di molecole con proteine).

1.1.2 ML vs. Programmazione Tradizionale

Nella programmazione tradizionale, il programmatore scrive esplicitamente le regole. Nel ML, le regole emergono automaticamente dai dati:



1.2 Il Framework di Apprendimento come Approssimazione di Funzione

Una visione unificante e formalmente rigorosa del ML è quella del **function approximation framework**: il problema di apprendimento è visto come l'approssimazione di una funzione sconosciuta f a partire da esempi.

1.2.1 I Cinque Elementi Fondamentali

Definizione 1.2: Framework ML in 5 elementi

Ogni problema di ML può essere descritto attraverso cinque elementi:

1. **DATA** $\mathcal{D}$: l'esperienza disponibile, rappresentata come vettori o strutture. Per il *supervised learning*: $\mathcal{D} = \{(\mathbf{x}^{(p)}, y^{(p)})\}_{p=1}^{\ell}$.
2. **TASK**: il tipo di problema (classificazione, regressione, non supervisionato, ...).
3. **MODEL**: descrive le relazioni tra i dati e definisce lo spazio di ipotesi $\mathcal{H}$ — l'insieme delle funzioni che il modello può implementare.
4. **LEARNING ALGORITHM**: dato il modello e i dati, esegue una ricerca (euristica) nello spazio $\mathcal{H}$ per trovare la miglior ipotesi h^* .
5. **VALIDATION**: valuta le capacità di generalizzazione dell'ipotesi trovata.

1.2.2 Notazione sui Dati

Simbolo	Significato
$\mathbf{x}^{(p)} \in \mathbb{R}^n$	p -esimo input (pattern/istanza/esempio)
$y^{(p)} \in \mathbb{R}$ (o classe)	Target/etichetta per il p -esimo esempio
ℓ	Numero totale di esempi nel training set
n	Dimensionalità dell'input (numero di feature)
$\mathbf{X} \in \mathbb{R}^{\ell \times n}$	Matrice dei dati: ℓ righe (pattern), n colonne (feature)
$x_{p,i}$	Componente i del p -esimo pattern

Convenzione: si usa notazione **bold per i vettori** ($\mathbf{x}, \mathbf{w}$) e **maiuscola bold per le matrici** ($\mathbf{X}, \mathbf{W}$). Gli scalari sono in minuscolo normale (x_i, w_j).

1.3 Tipologie di Task

1.3.1 Supervised Learning

Definizione 1.3: Supervised Learning

Dato: un training set di esempi **etichettati** $\mathcal{D} = \{(\mathbf{x}^{(p)}, y^{(p)})\}_{p=1}^{\ell}$ per una funzione target sconosciuta f .

Trovare: un'ipotesi **$h \in \mathcal{H}$ tale che $h \approx f$ con buona generalizzazione.**

Le due principali varianti sono:

- **Classificazione:** $f(\mathbf{x}) \in \{1, 2, \dots, K\}$ — output **discreto**. Es. riconoscimento di cifre scritte a mano, spam detection.
- **Regressione:** $f(\mathbf{x}) \in \mathbb{R}$ — output **continuo**. Es. predizione di prezzi, previsione meteorologica.

1.3.2 Unsupervised Learning

Il training set contiene **solo input** $\{\mathbf{x}^{(p)}\}_{p=1}^{\ell}$, **senza etichette**. L'obiettivo è **trovare strutture nei dati** (cluster, rappresentazioni compresse, distribuzioni). Esempi: k-means, PCA, autoencoders.

1.3.3 Altri tipi di apprendimento

- **Semi-supervised:** misto di esempi etichettati e non etichettati.
- **Reinforcement Learning:** l'agente impara attraverso **ricompense/ penalità** ottenute dall'interazione con l'**ambiente**.

- **Self-supervised**: le etichette vengono generate automaticamente dai dati stessi (es. pretraining dei Large Language Models).

1.4 Generalizzazione, Overfitting e Inductive Bias

1.4.1 Il Problema della Generalizzazione

L'obiettivo fondamentale del ML non è minimizzare l'errore sui dati di training, bensì costruire un modello che generalizza bene su nuovi dati.

Definizione 1.4: Inductive Learning Hypothesis

Qualunque ipotesi h che approssimi bene f sugli esempi di training approssimerà bene f anche su nuovi dati non visti.

Questa ipotesi, sebbene non dimostrabile in senso assoluto, è la base pratica del ML.

1.4.2 Overfitting

Definizione 1.5: Overfitting

Un modello **overfita** i dati se esiste un'ipotesi $h \in \mathcal{H}$ con errore empirico (training) E_{tr} e rischio (errore di generalizzazione) R , e un'altra ipotesi $h' \in \mathcal{H}$ con:

$$E_{tr}(h') > E_{tr}(h) \quad \text{ma} \quad R(h') < R(h)$$

ovvero h si adatta troppo ai dati di training perdendo capacità predittiva.

Esempio 1.1: Regressione polinomiale

Consideriamo un target $\sin(2\pi x)$ campionato con rumore gaussiano. Usando polinomi di grado M crescente:

- $M = 0$: **underfitting** — modello troppo semplice, bias alto.
- $M = 3$: buon **compromesso** — generalizza bene.
- $M = 9$ (con $\ell = 10$ punti): **overfitting** — errore training ≈ 0 ma generalizzazione pessima.

All'aumentare di M (o al diminuire di ℓ), il rischio di overfitting cresce.

1.4.3 Trade-off Bias-Variance

Il trade-off tra **bias** e **varianza** è centrale nel ML:

- **Bias alto** (underfitting): il modello è troppo **semplice** e non riesce a catturare la struttura dei dati.
- **Varianza alta** (overfitting): il modello è troppo **complesso** e si adatta al rumore del training set.

L'obiettivo è trovare il modello di **complessità ottimale** che minimizza l'errore di generalizzazione (si veda il Chapter 9 per il trattamento formale).

1.4.4 Inductive Bias

Definizione 1.6: Inductive Bias

L'**inductive bias** è l'insieme delle **assunzioni** che l'algoritmo di apprendimento fa a **priori** sulla forma della funzione target, **necessarie per generalizzare** da un training set finito.

Senza bias induttivo, la generalizzazione è impossibile (il problema è mal posto: infiniti h possono interpolare i dati). Esempi di bias induttivo:

- Preferire **ipotesi più semplici** (Rasoio di Occam).
- Assumere che i **dati vicini nello spazio di input abbiano output simili** (ipotesi di smoothness, usata da K-nn).
- Assumere **funzioni lineari** (modelli lineari).

1.5 Hypothesis Space e Parametri

Lo **spazio delle ipotesi** $\mathcal{H}$ definisce l'insieme delle funzioni che il modello può approssimare:

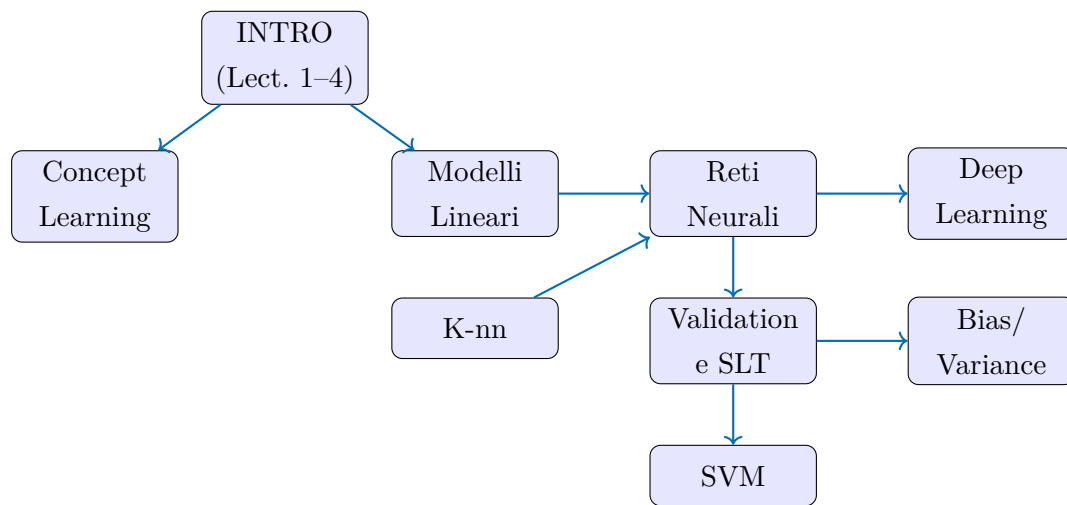
- **Finito discreto** $\mathcal{H}$: es. **concept learning** (alberi di decisione, regole logiche).
- **Continuo** $\mathcal{H}$: es. **modelli lineari, reti neurali, SVM** — lo spazio è parametrizzato da **pesi reali** $\mathbf{w} \in \mathbb{R}^d$.

Il **learning** algorithm esegue una **ricerca in $\mathcal{H}$ ottimizzando una funzione obiettivo** (loss function). La **difficoltà della ricerca** dipende da:

- La **natura della loss** (convessa vs. non convessa).
- La **dimensionalità di $\mathcal{H}$** (numero di parametri d).
- La **quantità e qualità dei dati** ℓ .

1.6 Struttura del Corso

Il corso segue la seguente progressione:



I modelli trattati coprono sia approcci *locali* (K-nn) che *globali* (reti neurali, SVM), sia ambienti *euclideo* che *strutturato* (grafi, sequenze).

Chapter 2

Modelli Lineari

2.1 Introduzione ai Modelli Lineari

I modelli lineari costituiscono la classe di ipotesi più **semplice** per funzioni in $\mathcal{H}$ *continuo*. Nonostante la semplicità, sono fondamentali perché:

- forniscono la **base teorica per le reti neurali** (il singolo neurone è un modello lineare con funzione di attivazione);
- sono **analiticamente trattabili**;
- **per dati linearmente separabili, la soluzione è ottimale**.

2.2 Linear Threshold Unit (LTU) e Perceptron

2.2.1 Il Modello

La **Linear Threshold Unit (LTU)** è il più semplice classificatore binario lineare. Data un'input $\mathbf{x} \in \mathbb{R}^n$, produce:

$$h_{\mathbf{w}}(\mathbf{x}) = \text{sign}(\mathbf{w}^T \mathbf{x} + w_0) = \text{sign}\left(\sum_{i=1}^n w_i x_i + w_0\right) \quad (2.1)$$

dove $\mathbf{w} = (w_1, \dots, w_n)^T \in \mathbb{R}^n$ sono i **pesi** (free parameters da apprendere) e w_0 è il **bias** (o threshold).

Nota 2.1: Convenzione sul bias

È conveniente **assorbire il bias nei pesi** aggiungendo una componente fittizia $x_0 = 1$ all'input: $\mathbf{w}^T \mathbf{x} = \sum_{i=0}^n w_i x_i$. In questo modo il vettore dei pesi include anche il bias.

2.2.2 Confine di Decisione

L'LTU definisce un **confine di decisione lineare** (iperpiano) nello spazio degli input:

$$\mathcal{B} = \{\mathbf{x} \mid \mathbf{w}^T \mathbf{x} = 0\}$$

Tutti i punti con $\mathbf{w}^T \mathbf{x} > 0$ vengono classificati come **+1**, quelli con $\mathbf{w}^T \mathbf{x} < 0$ come **-1**.

2.2.3 Algoritmo del Perceptron

L'**algoritmo del Perceptron** (Rosenblatt, 1958) è il primo algoritmo di **apprendimento** per **reti neurali**. Aggiorna i pesi ad ogni esempio mal classificato:

Algoritmo 1: Perceptron Learning Algorithm

1. **Inizializza** $\mathbf{w} \leftarrow \mathbf{0}$ (o valori piccoli casuali)
2. **repeat**
3. **for** ogni esempio $(\mathbf{x}^{(p)}, y^{(p)}) \in \mathcal{D}$ **do**
4. **if** $h_{\mathbf{w}}(\mathbf{x}^{(p)}) \neq y^{(p)}$ **then**
5. $\mathbf{w} \leftarrow \mathbf{w} + \eta \cdot y^{(p)} \cdot \mathbf{x}^{(p)}$
- end if**
- end for**
- until** tutti gli esempi sono classificati correttamente o max iterazioni

dove $\eta > 0$ è il **learning rate** (iperparametro).

Teorema 2.1: Convergenza del Perceptron

Se il training set è **linearmente separabile**, l'algoritmo del Perceptron converge in un numero finito di passi alla soluzione (un iperpiano separante). Se i dati *non* sono linearmente separabili, l'algoritmo non converge.

Limitazione fondamentale: il Perceptron può **rappresentare solo funzioni linearmente separabili**. Il famoso **controesempio** è la funzione **XOR**, non apprendibile da un singolo Perceptron (Minsky & Papert, 1969).

2.3 Regressione Lineare e Least Mean Squares (LMS)

2.3.1 Modello per Regressione

Per la **regressione** (output continuo), il **modello lineare** è:

$$h_{\mathbf{w}}(\mathbf{x}) = \mathbf{w}^T \mathbf{x} = \sum_{i=0}^n w_i x_i \quad (2.2)$$

2.3.2 Funzione di Errore (Loss)

L'**errore** da minimizzare è l'**errore quadratico medio** (Mean Squared Error):

$$E(\mathbf{w}) = \frac{1}{\ell} \sum_{p=1}^{\ell} \left(y^{(p)} - h_{\mathbf{w}}(\mathbf{x}^{(p)}) \right)^2 = \frac{1}{\ell} \|\mathbf{y} - \mathbf{X}\mathbf{w}\|^2 \quad (2.3)$$

dove $\mathbf{X} \in \mathbb{R}^{\ell \times n}$ è la matrice dei dati e $\mathbf{y} \in \mathbb{R}^{\ell}$ il vettore dei target.

2.3.3 Soluzione Analitica (Least Squares)

La funzione $E(\mathbf{w})$ è **convessa e quadratica in $\mathbf{w}$** , quindi ha un **minimo globale unico** (se $\mathbf{X}^T \mathbf{X}$ è invertibile):

$$\mathbf{w}^* = (\mathbf{X}^T \mathbf{X})^{-1} \mathbf{X}^T \mathbf{y} \quad (2.4)$$

La matrice $(\mathbf{X}^T \mathbf{X})^{-1} \mathbf{X}^T$ è detta **pseudo-inversa di Moore-Penrose** di $\mathbf{X}$.

Nota 2.2: Problemi della soluzione analitica

L'inversione di $\mathbf{X}^T \mathbf{X}$ ha complessità $O(n^3)$ e può essere numericamente **instabile**. Per n molto grande o dati ridondanti (collinearità), si usano **metodi iterativi** come il Gradient Descent.

2.3.4 Algoritmo LMS (Widrow-Hoff / Delta Rule)

L'**algoritmo LMS** (Least Mean Squares) è un algoritmo di **discesa del gradiente online** (o *stocastico*) **per minimizzare l'errore quadratico**:

$$\nabla_{\mathbf{w}} E = \frac{\partial E}{\partial \mathbf{w}} = \frac{2}{\ell} \sum_{p=1}^{\ell} \left(h_{\mathbf{w}}(\mathbf{x}^{(p)}) - y^{(p)} \right) \mathbf{x}^{(p)} \quad (2.5)$$

L'aggiornamento online (per il p -esimo esempio) è:

$$\Delta w_i = \eta \left(y^{(p)} - h_{\mathbf{w}}(\mathbf{x}^{(p)}) \right) x_i^{(p)} \quad \Leftrightarrow \quad \mathbf{w} \leftarrow \mathbf{w} + \eta \cdot \delta^{(p)} \cdot \mathbf{x}^{(p)} \quad (2.6)$$

dove $\delta^{(p)} = y^{(p)} - h_{\mathbf{w}}(\mathbf{x}^{(p)})$ è il **segnale di errore** (delta) per il pattern p .

LMS vs. Perceptron

La regola LMS è simile a quella del Perceptron, ma:

- LMS usa l'**errore reale** $\delta^{(p)}$ come segnale di aggiornamento.
- LMS **converge anche per dati non linearmente separabili** (trova il minimo dell'errore quadratico, che esiste sempre).
- LMS è usato per la **regressione**; il Perceptron per la **classificazione**.

2.4 Gradient Descent: Online, Batch e Mini-batch

La discesa del gradiente è il metodo di ottimizzazione fondamentale per i modelli parametrici. Esistono tre **varianti** principali:

1. **Batch Gradient Descent**: calcola il gradiente **sull'intero training set ad ogni passo**:

$$\mathbf{w} \leftarrow \mathbf{w} - \eta \cdot \nabla_{\mathbf{w}} E(\mathbf{w})$$

Pro: **convergenza stabile**; *Contro*: **lento per ℓ grande**.

2. **Stochastic (Online) Gradient Descent (SGD)**: aggiorna i pesi **dopo ogni singolo esempio**:

$$\mathbf{w} \leftarrow \mathbf{w} - \eta \cdot \nabla_{\mathbf{w}} E^{(p)}(\mathbf{w})$$

Pro: **veloce, può sfuggire a minimi locali**; *Contro*: **rumoroso**.

3. **Mini-batch Gradient Descent**: **compromesso** tra i due: **aggiorna dopo un piccolo sottoinsieme (mini-batch) di esempi**. *Pro*: **sfrutta il parallelismo hardware (GPU), stima del gradiente più stabile di SGD puro**.

2.5 Classificazione Lineare

2.5.1 Classificazione Binaria

Per la classificazione binaria si può usare la **regressione lineare** con **output soglia**:

$$h_{\mathbf{w}}(\mathbf{x}) = \begin{cases} 1 & \text{se } \mathbf{w}^T \mathbf{x} \geq 0.5 \\ 0 & \text{altrimenti} \end{cases}$$

Il **confine** di decisione $\{\mathbf{x} \mid \mathbf{w}^T \mathbf{x} = 0.5\}$ è un **iperpiano**.

2.5.2 Multiclasse (One-vs-All)

Per $K > 2$ classi, si possono addestrare K classificatori binari, uno per classe: il classificatore k -esimo impara a distinguere la classe k da tutte le altre. In fase di predizione, si sceglie la classe con output maggiore:

$$\hat{y} = \arg \max_{k \in \{1, \dots, K\}} \mathbf{w}_k^T \mathbf{x}$$

2.6 Limitazioni dei Modelli Lineari e Funzioni di Attivazione Non Lineari

I modelli lineari sono limitati alle funzioni *linearmente separabili*. Per superare questo limite esistono due approcci:

1. **Feature engineering**: trasformare l'input in uno spazio di feature non lineare $\phi(\mathbf{x})$ e applicare un modello lineare nel nuovo spazio.
2. **Reti neurali**: combinare più unità lineari con funzioni di attivazione non lineari per approssimare funzioni arbitrarie.

Questo motiva il passaggio ai modelli di reti neurali discussi nel Chapter 4.

Chapter 3

K-Nearest Neighbors (K-nn)

3.1 Introduzione: Approcci Lazy vs. Eager

Prima di descrivere il K-nn, introduciamo una distinzione fondamentale negli algoritmi di apprendimento basata su *quando* costruiscono l'ipotesi:

- **Eager learning**: il sistema *analizza i dati di training e costruisce esplicitamente un'ipotesi h* *prima di ricevere nuovi input* (es. reti neurali, SVM, modelli lineari).
- **Lazy learning**: il sistema *memorizza il training set* e *posticipa la costruzione dell'ipotesi al momento in cui si riceve un nuovo input da classificare*. Il K-nn è l'esempio canonico.

3.2 1-Nearest Neighbor (1-nn)

L'algoritmo più semplice è il 1-Nearest Neighbor:

Algoritmo 2: 1-Nearest Neighbor

1. **Training**: *memorizza tutti gli esempi $\{(\mathbf{x}^{(p)}, y^{(p)})\}_{p=1}^{\ell}$*
2. **Predizione** per un *nuovo input $\mathbf{x}$* :
3. $i^*(\mathbf{x}) \leftarrow \arg \min_{p \in \{1, \dots, \ell\}} d(\mathbf{x}, \mathbf{x}^{(p)})$
4. $\hat{y} \leftarrow y^{(i^*(\mathbf{x}))}$

dove $d(\mathbf{x}, \mathbf{x}^{(p)})$ è tipicamente la *distanza euclidea*:

$$d(\mathbf{x}, \mathbf{x}^{(p)}) = \|\mathbf{x} - \mathbf{x}^{(p)}\| = \sqrt{\sum_{t=1}^n (x_t - x_{p,t})^2}$$

3.2.1 Proprietà del 1-nn

- **Errore di training = 0**: il 1-nn classifica correttamente tutti gli esempi di training (ogni punto è il suo stesso vicino più prossimo).
- **Confine di decisione non lineare**: il confine è determinato dal **diagramma di Voronoi**, che assegna ad ogni punto dello spazio la classe del punto di training più vicino.
- **Alta varianza**: il modello si adatta fortemente al rumore nei dati.

3.2.2 Diagramma di Voronoi

Il diagramma di Voronoi partiziona $\mathbb{R}^n$ in celle, ciascuna contenente tutti i punti più vicini a un dato training point $\mathbf{x}^{(p)}$ che a qualsiasi altro:

$$V_p = \{\mathbf{x} \in \mathbb{R}^n \mid d(\mathbf{x}, \mathbf{x}^{(p)}) \leq d(\mathbf{x}, \mathbf{x}^{(q)}) \forall q \neq p\}$$

3.3 K-Nearest Neighbors

3.3.1 Generalizzazione a K Vicini

Il **K-nn** introduce uno *smoothing* sulle predizioni considerando i K vicini più prossimi $\mathcal{N}_K(\mathbf{x})$:

$$\hat{y}_K(\mathbf{x}) = \frac{1}{K} \sum_{\mathbf{x}^{(p)} \in \mathcal{N}_K(\mathbf{x})} y^{(p)} \quad (3.1)$$

- **Classificazione**: *majority voting* tra i K vicini.

$$h(\mathbf{x}) = \begin{cases} +1 & \text{se } \hat{y}_K(\mathbf{x}) \geq 0.5 \\ -1 & \text{altrimenti} \end{cases}$$

- **Regressione**: usa direttamente la media $\hat{y}_K(\mathbf{x})$.

3.3.2 Effetto di K

- **$K = 1$** : massima *flessibilità*, alto *rischio di overfitting* su dati rumorosi. Errore training = 0.
- **K grande**: maggiore *smoothing*, *confine di decisione più regolare*, ma *rischio di underfitting* se K è troppo grande.

- $K = \ell$: predice sempre la classe maggioritaria nel training set (classificatore banale).

Il parametro K è un *iperparametro* da selezionare tramite validation (si veda il Chapter 6).

3.4 Misure di Distanza

La scelta della distanza è cruciale per K-nn. Le principali metriche sono:

Metrica	Formula	Note
Euclidea (L_2)	$\sqrt{\sum_i (x_i - x'_i)^2}$	Più comune
Manhattan (L_1)	$\sum_i x_i - x'_i $	Robusta agli outlier
Minkowski (L_p)	$(\sum_i x_i - x'_i ^p)^{1/p}$	Generalizzazione
Mahalanobis	$\sqrt{(\mathbf{x} - \mathbf{x}')^T \Sigma^{-1} (\mathbf{x} - \mathbf{x}')}$	Considera correlazioni

3.5 Analisi Teorica: Curse of Dimensionality

In alta dimensionalità (n grande), il K-nn soffre della *curse of dimensionality*:

- Il volume di una sfera di raggio r in $\mathbb{R}^n$ cresce come r^n ; per catturare una frazione fissa del training set, il raggio necessario cresce esponenzialmente con n .
- La distanza tra il punto più vicino e il più lontano diventa sempre più simile: i vicini sono "ugualmente lontani".
- Servono esponenzialmente più dati per mantenere la stessa densità locale.

Nota 3.1: Risultato asintotico del 1-nn

Quando $\ell \rightarrow \infty$ (infiniti esempi), il tasso di errore del 1-nn soddisfa:

$$P^* \leq P_{1nn} \leq 2P^*(1 - P^*)$$

dove P^* è il *Bayes error rate* (errore minimo teoricamente raggiungibile). Pertanto, il 1-nn con dati infiniti ha errore al più il doppio del Bayes error.

3.6 Proprietà e Limiti Pratici

Vantaggi

- **Semplicità**: nessuna fase di training esplicita.

- **Flessibilità**: può approssimare confini di decisione arbitrariamente complessi.
- **Nessuna assunzione sulla forma della distribuzione dei dati**.

Svantaggi

- **Complessità computazionale** in predizione: $O(\ell \cdot n)$ per trovare i K vicini (possibile mitigazione con strutture come KD-tree).
- **Memoria**: richiede di memorizzare tutti gli esempi di training.
- **Curse of dimensionality**: performance degrada per n grande.
- **Sensibilità alla scala delle feature**: feature con range diversi dominano la distanza $\Rightarrow$ è necessaria la *normalizzazione*.

Inductive Bias del K-nn

Il K-nn assume che punti vicini nello spazio degli input abbiano output simili (*smoothness assumption*). Questo è un bias **debole** (non specifico sulla forma funzionale), ma che soffre in alta dimensionalità.

3.7 Confronto: Modelli Locali vs. Globali

	Locale (K-nn)	Globale (Lineare/NN)
Fase di training	Nessuna	Ottimizzazione dei pesi
Fase di predizione	$O(\ell \cdot n)$	$O(n)$
Capacità	Arbitraria (con K piccolo)	Dipende dal modello
Inductive bias	Smoothness locale	Dipende dal modello
Parametri espliciti	Nessuno (solo K)	w

Chapter 4

Reti Neurali Artificiali — Parte 1

4.1 Motivazione e Obiettivi

Le **Reti Neurali Artificiali** (ANN) sono modelli ispirate al sistema nervoso biologico, utilizzate come strumenti **flessibili** di Machine Learning. Per il corso:

- Le ANN sono **Universal Approximators**: possono approssimare qualsiasi funzione continua su un compatto con precisione arbitraria (Teorema di Cybenko, 1989).
- Sono **robuste al rumore e ai dati mancanti**.
- Gestiscono sia **task di classificazione che di regressione**.
- Costituiscono un **paradigma**: comprendono un'ampia famiglia di modelli (MLP, CNN, RNN, Transformers, ...).

4.2 Il Neurone Biologico e il Modello Artificiale

4.2.1 Neurone Biologico

Il neurone biologico è l'unità di calcolo del sistema nervoso:

- **Dendrites**: rami in ingresso che ricevono segnali dagli altri neuroni.
- **Corpo cellulare (soma)**: integra i segnali in ingresso.
- **Assone**: trasmette il segnale di uscita ad altri neuroni.
- **Sinapsi**: giunzione tra neuroni; la sua forza (peso sinaptico) può variare con l'apprendimento (*plasticità sinaptica*).

Hebbian learning (Hebb, 1949): "Neurons that fire together, wire together" — la forza di una sinapsi aumenta quando i neuroni collegati sono attivi contemporaneamente.

4.2.2 Neurone Artificiale (Processing Unit)

L'unità artificiale modella il comportamento del neurone biologico:

$$\text{net}_i(\mathbf{x}) = \sum_{j=0}^n w_{ij}x_j = \mathbf{w}_i^T \mathbf{x} \quad (4.1)$$

$$o_i(\mathbf{x}) = f(\text{net}_i(\mathbf{x})) \quad (4.2)$$

dove:

- x_j : j -esimo input (con $x_0 = 1$ per il bias).
- w_{ij} : peso della connessione da j all'unità i (parametro libero).
- net_i : **net input** — combinazione lineare pesata degli ingressi.
- $f(\cdot)$: **funzione di attivazione** — introduce la non linearità.
- o_i : output del neurone i .

4.3 Funzioni di Attivazione

La funzione di attivazione f determina il **tipo di neurone** e le sue proprietà:

4.3.1 Funzione a Gradino (**Step**/Heaviside)

$$f(\text{net}) = \begin{cases} 1 & \text{se } \text{net} \geq 0 \\ 0 & \text{altrimenti} \end{cases}$$

Usata nel **Perceptron**. *Problema*: **non differenziabile in 0 $\Rightarrow$ non adatta per gradient descent.**

4.3.2 Funzione Lineare (**Identità**)

$$f(\text{net}) = \text{net}$$

Usata per la **regressione** (output layer). Porta a modelli lineari.

4.3.3 **Sigmoid** Logistica

$$f(\text{net}) = \sigma(\text{net}) = \frac{1}{1 + e^{-\text{net}}} \in (0, 1) \quad (4.3)$$

Proprietà:

- Output **limitato in (0, 1)**: interpretabile come **probabilità**.

- Differenziabile ovunque con derivata semplice: $\sigma'(net) = \sigma(net)(1 - \sigma(net))$.
- Problema: *vanishing gradient* per $|net|$ molto grande.

4.3.4 Tangente Iperbolica (**tanh**)

$$f(net) = \tanh(net) = \frac{e^{net} - e^{-net}}{e^{net} + e^{-net}} \in (-1, 1) \quad (4.4)$$

Vantaggi rispetto alla sigmoide: **output centrato in zero** (migliora la convergenza).
Vale: $\tanh(net) = 2\sigma(2 \cdot net) - 1$.

4.3.5 **ReLU** (Rectified Linear Unit)

$$f(net) = \max(0, net) = \begin{cases} net & \text{se } net > 0 \\ 0 & \text{altrimenti} \end{cases} \quad (4.5)$$

Attualmente la più usata nel **deep learning**:

- Computazionalmente **efficiente**.
- **Non soffre di vanishing gradient** per valori positivi.
- **Problema: "dying ReLU"** (unità con output sempre zero).

4.3.6 Softmax (per output layer multiclasse)

$$o_k = \text{softmax}(\mathbf{net})_k = \frac{e^{net_k}}{\sum_{j=1}^K e^{net_j}} \quad (4.6)$$

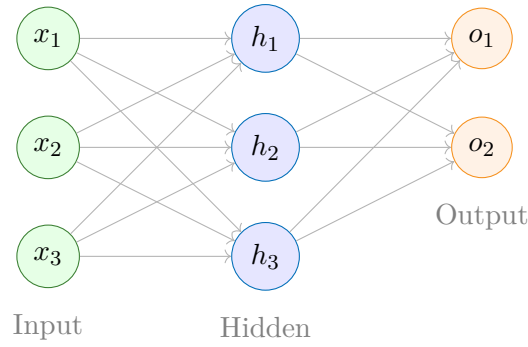
Produce una **distribuzione di probabilità su K classi**: $\sum_k o_k = 1$.

4.4 Multi-Layer Perceptron (**MLP**)

4.4.1 Architettura

Il **Multi-Layer Perceptron** è una rete neurale **feedforward** (a propagazione in avanti) composta da:

1. **Input layer**: n nodi, uno per feature (nessuna computazione).
2. **Hidden layer(s)**: uno o più strati nascosti con unità non lineari.
3. **Output layer**: K unità (una per classe o una per regressione).



4.4.2 Forward Pass

Per un MLP con un hidden layer, il **forward pass** (propagazione in avanti) è:

$$\text{Hidden layer: } a_j = f^{(1)}\left(\sum_{i=0}^n w_{ji}^{(1)} x_i\right) = f^{(1)}(\text{net}_j^{(1)}) \quad (4.7)$$

$$\text{Output layer: } o_k = f^{(2)}\left(\sum_{j=0}^m w_{kj}^{(2)} a_j\right) = f^{(2)}(\text{net}_k^{(2)}) \quad (4.8)$$

dove m è il numero di unità hidden e $f^{(1)}, f^{(2)}$ sono le funzioni di attivazione del hidden e output layer rispettivamente.

4.4.3 Teorema di Approssimazione Universale

Teorema 4.1: Universal Approximation (Cybenko, 1989)

Un MLP con un **singolo hidden layer** contenente un numero **finito** di neuroni con funzione di attivazione **sigmoide** può approssimare **qualsiasi funzione continua** $f : \mathbb{R}^n \rightarrow \mathbb{R}$ su un compatto con precisione arbitraria $\varepsilon > 0$.

Nota importante: il teorema garantisce l'esistenza di tale rete, ma non fornisce un algoritmo per trovarla. In pratica, il numero di neuroni necessario può essere esponenzialmente grande $\Rightarrow$ motivazione per il *deep learning*.

4.5 Funzione Obiettivo e Task

4.5.1 Regressione: **Sum-of-Squares Error**

$$E(\mathbf{w}) = \frac{1}{2} \sum_{p=1}^{\ell} \sum_{k=1}^K \left(y_k^{(p)} - o_k^{(p)} \right)^2 \quad (4.9)$$

4.5.2 Classificazione: Cross-Entropy Loss

Per **classificazione binaria** (con sigmoide nell'output):

$$E(\mathbf{w}) = - \sum_{p=1}^{\ell} \left[y^{(p)} \log o^{(p)} + (1 - y^{(p)}) \log(1 - o^{(p)}) \right] \quad (4.10)$$

Per classificazione **multiclasse** (con softmax):

$$E(\mathbf{w}) = - \sum_{p=1}^{\ell} \sum_{k=1}^K y_k^{(p)} \log o_k^{(p)} \quad (4.11)$$

4.6 Rappresentazione: Cosa Può Imparare un MLP?

L'aggiunta di **layer nascosti** aumenta drasticamente la **capacità rappresentativa**:

- **Perceptron (0 hidden layer)**: solo funzioni **linearmente separabili**.
- **MLP con 1 hidden layer**: funzioni con **confini di decisione arbitrari** (approssimazione universale su compatti).
- **MLP con 2 hidden layer**: funzioni con **qualsiasi forma** (anche discontinue, con sufficiente larghezza).

Esempio 4.1: XOR con MLP

La funzione **XOR** non è linearmente separabile. Un MLP con 2 unità hidden può realizzarla esattamente: il primo layer nascosto apprende le "primitive" AND e OR, il secondo le combina per ottenere XOR. Con pesi appropriati:

$$h_1 = \text{sign}(x_1 + x_2 - 1.5) \quad (\text{AND})$$

$$h_2 = \text{sign}(x_1 + x_2 - 0.5) \quad (\text{OR})$$

$$o = \text{sign}(-h_1 + h_2 - 0.5) \quad (\text{XOR})$$

4.7 Gradient Descent per MLP: Introduzione alla Backpropagation

Per addestrare un MLP si usa il **gradient descent** sulla loss $E(\mathbf{w})$. La **difficoltà** rispetto ai modelli lineari è il **calcolo del gradiente per i pesi degli hidden layer**, poiché l'errore non è direttamente osservabile lì.

La soluzione è l'**algoritmo di backpropagation** (retropropagazione dell'errore), trattato in dettaglio nel Chapter 5. L'idea chiave è **applicare** la **regola della catena** (chain rule) per **propagare il segnale di errore dall'output verso i layer nascosti**.

Chapter 5

Reti Neurali Artificiali — Parte 2: Backpropagation e Training

5.1 L'Algoritmo di Backpropagation

La **backpropagation** (retropropagazione dell'errore) è l'algoritmo fondamentale per addestrare un MLP. Combina due passaggi:

1. **Forward pass**: calcola l'output della rete dato l'input.
2. **Backward pass**: propaga il segnale di errore dall'output agli hidden layer usando la regola della catena, calcolando i gradienti rispetto ai pesi.

5.1.1 Derivazione per l'Output Layer

Consideriamo la **loss Sum-of-Squares (SSE)** su un pattern p :

$$E^{(p)} = \frac{1}{2} \sum_k (y_k^{(p)} - o_k^{(p)})^2$$

Il **gradiente rispetto ai pesi** del layer di output $w_{kj}^{(2)}$:

$$\frac{\partial E^{(p)}}{\partial w_{kj}^{(2)}} = \underbrace{(o_k^{(p)} - y_k^{(p)})}_{-\delta_k^{out}} \cdot f'(\text{net}_k^{(2)}) \cdot a_j^{(p)} \quad (5.1)$$

Definendo il **segnale di errore** per l'output layer:

$$\delta_k^{out} = f'(\text{net}_k^{(2)}) \cdot (y_k^{(p)} - o_k^{(p)}) \quad (5.2)$$

L'aggiornamento dei pesi di output è:

$$\Delta w_{kj}^{(2)} = \eta \cdot \delta_k^{out} \cdot a_j^{(p)}$$

5.1.2 Derivazione per i Layer Nascosti

Per un hidden unit j non è direttamente osservabile l'errore. Si usa la **chain rule**:

$$\frac{\partial E^{(p)}}{\partial w_{ji}^{(1)}} = \frac{\partial E^{(p)}}{\partial \text{net}_j^{(1)}} \cdot \frac{\partial \text{net}_j^{(1)}}{\partial w_{ji}^{(1)}} = \delta_j^{\text{hidden}} \cdot x_i^{(p)} \quad (5.3)$$

Il segnale di errore per l'hidden unit j è ottenuto *retropropagando* l'errore dall'output:

$$\delta_j^{\text{hidden}} = f'(\text{net}_j^{(1)}) \cdot \sum_k \delta_k^{\text{out}} \cdot w_{kj}^{(2)} \quad (5.4)$$

Il termine $\sum_k \delta_k^{\text{out}} \cdot w_{kj}^{(2)}$ è l'errore "proveniente" dai neuroni di output verso il neurone nascosto j (pesato dai pesi del layer di output).

5.1.3 Algoritmo Completo

Algoritmo 3: Backpropagation (versione **online**/stocastica)

1. **Inizializza** pesi $\mathbf{w}$ con valori piccoli casuali
2. **repeat**
3. **for** ogni pattern $(\mathbf{x}^{(p)}, \mathbf{y}^{(p)}) \in \mathcal{D}$ **do**
4. **Forward** pass: calcola $\text{net}^{(l)}, o^{(l)}$ per ogni layer l
5. **Output**: calcola $\delta_k^{\text{out}} = f'(\text{net}_k) \cdot (y_k - o_k)$
6. **Backward**: per $l = L - 1, L - 2, \dots, 1$:

$$\delta_j^{(l)} = f'(\text{net}_j^{(l)}) \cdot \sum_k \delta_k^{(l+1)} w_{kj}^{(l+1)}$$
7. **Update**: $w_{ji}^{(l)} \leftarrow w_{ji}^{(l)} + \eta \cdot \delta_j^{(l)} \cdot o_i^{(l-1)}$
- end for**
- until** criterio di stop

5.1.4 Derivata delle Funzioni di Attivazione

Per implementare la backpropagation serve $f'(\text{net})$:

Funzione	$f(\text{net})$	$f'(\text{net})$
Sigmoide	$\sigma(\text{net})$	$\sigma(\text{net})(1 - \sigma(\text{net}))$
Tanh	$\tanh(\text{net})$	$1 - \tanh^2(\text{net})$
ReLU	$\max(0, \text{net})$	$\mathbf{1}[\text{net} > 0]$
Lineare	net	1

5.2 Euristiche e Iperparametri per il Training

Addestrare un MLP implica numerose scelte progettuali (iperparametri) che influenzano la qualità dell'apprendimento.

5.2.1 Inizializzazione dei Pesi

- **Non inizializzare a zero**: tutti i neuroni di uno stesso layer avrebbero gli stessi gradienti (simmetry breaking problem).
- **Inizializzazione casuale uniforme**: $w_{ij} \sim \mathcal{U}[-\epsilon, \epsilon]$ con ϵ piccolo (es. 0.1).
- **Xavier/Glorot**: $w_{ij} \sim \mathcal{U}\left[-\frac{\sqrt{6}}{\sqrt{n_{in}+n_{out}}}, \frac{\sqrt{6}}{\sqrt{n_{in}+n_{out}}}\right]$ per sigmoide/tanh.
- **He**: $w_{ij} \sim \mathcal{N}(0, 2/n_{in})$ per ReLU.

5.2.2 Learning Rate η

Il learning rate è l'iperparametro più **critico**:

- **Troppo grande**: oscillazioni o divergenza.
- **Troppo piccolo**: convergenza lenta.
- **Adaptive learning rate**: metodi come Adam, RMSProp adattano η per ogni **peso** (si veda il Chapter 11).

Learning rate schedule: **ridurre η nel tempo migliora la convergenza finale**.

5.2.3 Online vs. Batch vs. Mini-batch

	Online (SGD)	Mini-batch	Batch
Gradiente	Singolo esempio	Sottoinsieme	Tutto il dataset
Rumore	Alto	Medio	Basso
Velocità/epoca	Lento	Medio	Veloce (GPU)
Convergenza	Instabile	Buona	Stabile

5.2.4 Momentum

Il momentum **accelera la convergenza** aggiungendo una componente "inerziale":

$$\Delta \mathbf{w}^{(t)} = -\eta \nabla E + \alpha \Delta \mathbf{w}^{(t-1)} \quad (5.5)$$

dove $\alpha \in [0, 1)$ è il coefficiente di momentum. Aiuta a:

- Accelerare la discesa in direzioni consistenti.
- Smorzare le oscillazioni nelle direzioni di alta curvatura.

5.2.5 Criteri di Stop

- Numero **massimo di epoche**.
- **Soglia sull'errore training** (non raccomandato da solo: rischio overfitting).
- **Early stopping**: monitorare l'errore su un validation set e fermare quando smette di migliorare (tecnica di regolarizzazione implicita).

5.3 Minimi Locali e Non-Convessità

La **loss di un MLP è in generale non convessa**: esistono **multipli minimi locali, punti di sella, e plateau**. Tuttavia:

- In pratica, molti minimi locali hanno **performance simili** (specialmente in reti profonde — "loss landscape" quasi piatto).
- Il SGD con **rumore** può aiutare a **sfuggire da minimi piatti/locali**.
- **La non-convessità è meno problematica in alta dimensionalità** (i punti di sella dominano, ma il gradiente li evita statisticamente).

5.4 Overfitting e Regolarizzazione

Le reti neurali sono tipicamente **over-parametrizzate** (molti più pesi che esempi di training), con alto **rischio di overfitting**.

5.4.1 Weight Decay (Regolarizzazione L2)

Aggiunge un **termine di penalizzazione alla loss per mantenere i pesi piccoli**:

$$E_{reg}(\mathbf{w}) = E(\mathbf{w}) + \frac{\lambda}{2} \|\mathbf{w}\|^2 \quad (5.6)$$

dove $\lambda > 0$ è il coefficiente di regolarizzazione. L'aggiornamento diventa:

$$\Delta w_{ij} = \eta \delta_j o_i - \eta \lambda w_{ij}$$

5.4.2 Early Stopping

1. Dividere i dati in training e validation set.
2. Monitorare l'errore di validation ad ogni epoca.
3. Fermare il training quando l'errore di validation ricomincia a salire.
4. Usare i pesi corrispondenti al minimo di validation.

Perché l'early stopping funziona?

Limitare il numero di epoche è equivalente a limitare la capacità effettiva del modello: i pesi non hanno tempo di crescere troppo, riducendo la varianza. L'early stopping è una forma di regolarizzazione implicita.

5.4.3 Numero di Hidden Units

Il numero di unità nascoste m controlla la capacità del modello:

- m piccolo: underfitting (bias alto).
- m grande: rischio overfitting (varianza alta), ma con regolarizzazione può generalizzare bene.

Tipicamente si sceglie m via cross-validation.

5.4.4 Dropout (Cenni)

Il dropout (Srivastava et al., 2014) disattiva casualmente una frazione p di neuroni ad ogni passo di training (sostituisce i pesi con zero). Equivale ad allenare un ensemble di sottoreti. Si veda il Chapter 11.

5.5 Rappresentazione dell'Input e dell'Output

5.5.1 Normalizzazione dell'Input

- Standardizzazione (z-score): $x'_i = (x_i - \mu_i) / \sigma_i$ (media zero, deviazione standard 1).

- **Min-Max scaling**: $x'_i = (x_i - x_{min}) / (x_{max} - x_{min}) \in [0, 1]$.

Motivazione: evita che feature con scale diverse dominino i gradienti.

5.5.2 Codifica dell'Output

- **Classificazione binaria**: 1 output con sigmoide, target in $\{0, 1\}$.
- **Classificazione K-classe**: K output con softmax e cross-entropy, target come one-hot vector $(0, \dots, 1, \dots, 0)$.
- **Regressione**: 1 output lineare, target scalare.

5.6 Cascade Correlation (Cenni)

Cascade Correlation (Fahlman & Lebiere, 1990) è un algoritmo costruttivo: parte da una rete senza hidden unit e aggiunge neuroni one-by-one. Ogni nuovo neurone viene addestrato per massimizzare la correlazione tra il suo output e il residuo di errore. Vantaggi: determina automaticamente la dimensione della rete; non soffre del vanishing gradient nei layer profondi.

Chapter 6

Model Assessment e Model Selection

6.1 Introduzione: Il Problema della Valutazione

Una volta addestrato un modello, occorre rispondere a due domande distinte:

- **Model Selection**: quale modello (o quale configurazione di iperparametri) scegliere? Si confrontano diversi modelli per trovare il migliore.
- **Model Assessment**: quanto bene generalizza il modello scelto? Si stima il rischio (errore di generalizzazione) sul modello finale.

Distinzione fondamentale

Model selection e model assessment richiedono **insiemi di dati separati**: usare gli stessi dati per selezionare e valutare porta a una stima *ottimistica* (bias nella valutazione). In pratica: **training set, validation set, test set**.

6.2 Definizioni Formali di Errore

6.2.1 Training Error (Errore Empirico)

$$E_{tr}(h) = \frac{1}{\ell_{tr}} \sum_{p=1}^{\ell_{tr}} \mathcal{L}(y^{(p)}, h(\mathbf{x}^{(p)})) \quad (6.1)$$

L'errore di training misura quanto il modello si adatta ai dati visti. Con modelli flessibili, $E_{tr} \rightarrow 0$ (overfitting): pertanto **non è una buona stima dell'errore di generalizzazione**.

6.2.2 Generalization Error (Rischio)

$$R(h) = \mathbb{E}_{(\mathbf{x}, y) \sim P}[\mathcal{L}(y, h(\mathbf{x}))] \quad (6.2)$$

Il rischio è il valore atteso della loss sulla distribuzione vera dei dati P — la quantità che vogliamo minimizzare ma non possiamo osservare direttamente.

6.2.3 Test Error

Il test error è una stima del rischio su dati non usati per training o selection:

$$E_{ts}(h) = \frac{1}{\ell_{ts}} \sum_{p=1}^{\ell_{ts}} \mathcal{L}(y^{(p)}, h(\mathbf{x}^{(p)}))$$

Regola d'oro: il test set si usa una sola volta, dopo aver completato la model selection, per avere una stima non distorta del rischio.

6.3 Hold-Out e Splitting dei Dati

6.3.1 Schema a Tre Insiemi

Training Set (60%–70%)	Validation (15%–20%)	Test (15%–20%)
------------------------	----------------------	----------------

- **Training set:** usato per addestrare il modello (ottimizzare i pesi).
- **Validation set:** usato per la model selection (scegliere iperparametri, early stopping). Può essere visto più volte.
- **Test set:** usato una sola volta per la valutazione finale.

Problema: con pochi dati, il hold-out singolo ha alta varianza nella stima.

6.4 Cross-Validation

La **cross-validation** (CV) è la tecnica standard per stimare il rischio e/o per la model selection quando i dati sono limitati.

6.4.1 K-Fold Cross-Validation

Algoritmo 4: K-Fold Cross-Validation

1. Partiziona $\mathcal{D}$ in K fold $\mathcal{D}_1, \dots, \mathcal{D}_K$ di dimensione approssimativamente uguale
2. **for** $k = 1, 2, \dots, K$ **do**
3. **Addestra** il modello su $\mathcal{D} \setminus \mathcal{D}_k$
4. Valuta l'errore E_k su $\mathcal{D}_k$
- end for**
5. $\hat{R}_{CV} = \frac{1}{K} \sum_{k=1}^K E_k$

- $K = 5$ o $K = 10$: scelta comune, buon compromesso bias-varianza.
- $K = \ell$ (**Leave-One-Out, LOO**): ogni fold contiene un solo esempio; stima quasi unbiased ma alta varianza e computazionalmente costosa.

6.4.2 Uso Corretto della Cross-Validation

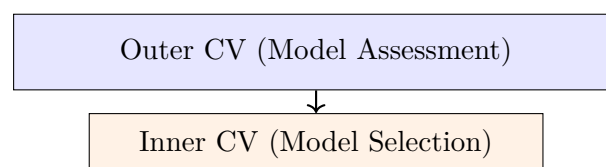
Come usare la CV correttamente

1. **Model selection**: usare CV sul training set per scegliere gli iperparametri. Il validation set interno alla CV serve per la selezione.
2. **Riaddestramento**: dopo la selezione, riaddestrare il modello su **tutto il training set** con gli iperparametri ottimali.
3. **Assessment**: valutare il modello finale sul test set (esterno alla CV).

Non usare mai i dati di test nella CV!

6.4.3 Schema Nested Cross-Validation

Per una valutazione rigorosa con pochi dati:



La CV esterna stima il rischio; la CV interna seleziona gli iperparametri.

6.5 Bias e Varianza della Stima dell'Errore

Definizione 6.1: Bias e Varianza della stima

Sia $\hat{R}$ la stima del rischio R . Definiamo:

- **Bias:** $\mathbb{E}[\hat{R}] - R$ — errore sistematico (quanto la stima è mediamente sopra/sotto il vero rischio).
- **Varianza:** $\mathbb{E}[(\hat{R} - \mathbb{E}[\hat{R}])^2]$ — instabilità della stima al variare del training set.

Metodo	Bias	Varianza	Costo
Training error	Pessimistico (negativo)	Bassa	$O(1)$
Hold-out (small)	Pessimistico	Alta	$O(1)$ train
K -fold CV ($K = 5, 10$)	Leggermente pessimistico	Media	$O(K)$ train
LOOCV	Quasi unbiased	Alta	$O(\ell)$ train
Bootstrap	Pessimistico (correggibile)	Bassa	$O(B)$ train

6.6 Bootstrap

Il **bootstrap** è un metodo di ricampionamento che stima la distribuzione di un estimatore:

1. Genera B campioni bootstrap $\mathcal{D}^{*1}, \dots, \mathcal{D}^{*B}$ campionando con ripetizione da $\mathcal{D}$.
2. Addestra il modello su ogni $\mathcal{D}^{*b}$ e valuta sugli esempi non inclusi (out-of-bag, OOB).
3. Aggrega le stime.

Stimatore .632: corregge il bias del bootstrap puro:

$$\hat{R}_{.632} = 0.368 \cdot E_{tr} + 0.632 \cdot E_{OOB}$$

(il 36.8% degli esempi in media non appaiono in un campione bootstrap).

6.7 Metriche di Valutazione

6.7.1 Classificazione

Metrica	Formula
Accuracy	$(TP + TN)/(TP + TN + FP + FN)$
Error rate	$1 - \text{Accuracy}$
Precision	$TP/(TP + FP)$
Recall (Sensitivity)	$TP/(TP + FN)$
F1 score	$2 \cdot (P \cdot R)/(P + R)$
AUC-ROC	Area sotto la curva ROC

- **TP**: True Positives; **TN**: True Negatives; **FP**: False Positives; **FN**: False Negatives.
- **Confusion matrix**: tabella $K \times K$ con le predizioni vs. i veri label per la classificazione multiclasse.

6.7.2 Regressione

Metrica	Formula
MSE	$\frac{1}{\ell} \sum_p (y^{(p)} - \hat{y}^{(p)})^2$
RMSE	$\sqrt{MSE}$
MAE	$\frac{1}{\ell} \sum_p y^{(p)} - \hat{y}^{(p)} $
R^2	$1 - SS_{res}/SS_{tot}$

6.8 Selezione del Modello: AIC e BIC

Per modelli lineari con distribuzione gaussiana degli errori, esistono criteri analitici per la model selection:

$$\text{AIC} = -2 \log \mathcal{L}(\hat{\mathbf{w}}) + 2d \quad (6.3)$$

$$\text{BIC} = -2 \log \mathcal{L}(\hat{\mathbf{w}}) + d \log \ell \quad (6.4)$$

dove d è il numero di parametri, ℓ il numero di esempi e $\mathcal{L}$ la verosimiglianza. Il BIC penalizza i modelli complessi più dell'AIC (per $\ell > 7$). Si sceglie il modello con AIC/BIC minore.

Chapter 7

Statistical Learning Theory e VC Dimension

7.1 Motivazione: Basi Teoriche della Generalizzazione

La **Statistical Learning Theory** (SLT), sviluppata principalmente da Vapnik e Chervonenkis, fornisce **garanzie teoriche** sulla generalizzazione. L'obiettivo è rispondere alla domanda: *quanti dati servono per garantire che l'errore di training sia una buona approssimazione del rischio?*

7.2 Setup Formale

- Distribuzione sconosciuta $P(\mathbf{x}, y)$ su $\mathcal{X} \times \mathcal{Y}$.
- Training set $\mathcal{D} = \{(\mathbf{x}^{(p)}, y^{(p)})\}_{p=1}^{\ell}$ **i.i.d.** da P .
- Spazio di ipotesi $\mathcal{H}$.
- Errore empirico: $E_{tr}(h) = \frac{1}{\ell} \sum_p \mathbf{1}[h(\mathbf{x}^{(p)}) \neq y^{(p)}]$.
- Rischio: $R(h) = P(h(\mathbf{x}) \neq y)$.

7.3 VC Dimension

Definizione 7.1: Shattering

Si dice che $\mathcal{H}$ **frammenta** (*shatters*) un insieme di punti $S = \{\mathbf{x}_1, \dots, \mathbf{x}_N\}$ se per ogni possibile etichettatura (dicotomia) binaria di S esiste un'ipotesi $h \in \mathcal{H}$ che le realizza correttamente. Ci sono 2^N possibili dicotomie su N punti.

Definizione 7.2: VC Dimension

La **VC dimension** (Vapnik-Chervonenkis dimension) di $\mathcal{H}$ è il massimo numero d_{VC} di punti che $\mathcal{H}$ può frammentare:

$$d_{VC}(\mathcal{H}) = \max\{N : \exists S \subset \mathcal{X}, |S| = N, \mathcal{H} \text{ frammenta } S\}$$

7.3.1 Esempi di VC Dimension

Modello	d_{VC}
Classificatore costante	0
Iperpiani in $\mathbb{R}^n$ (senza bias)	n
Iperpiani in $\mathbb{R}^n$ (con bias)	$n + 1$
Funzioni a gradino in $\mathbb{R}^1$	1
Intervalli in $\mathbb{R}^1$	2
Convex sets in $\mathbb{R}^2$	∞

Esempio 7.1: VC Dimension di iperpiani in $\mathbb{R}^2$

In $\mathbb{R}^2$, un iperpiano (retta) ha $d_{VC} = 3$: si possono frammentare 3 punti (qualsiasi etichettatura di 3 punti non collineari è realizzabile da una retta), ma non 4 (la configurazione XOR non è separabile).

7.4 Structural Risk Minimization (SRM)**Teorema 7.1: Limite VC (Vapnik-Chervonenkis Bound)**

Con probabilità almeno $1 - \delta$ su un training set di ℓ esempi, per ogni $h \in \mathcal{H}$ vale:

$$R(h) \leq E_{tr}(h) + \sqrt{\frac{d_{VC}(\ln(2\ell/d_{VC}) + 1) - \ln(\delta/4)}{\ell}} \quad (7.1)$$

dove il secondo termine è detto **Complexity Penalty**.

Il bound mostra che:

- $\ell \uparrow$: il bound si stringe (più dati $\Rightarrow$ migliore generalizzazione).
- $d_{VC} \uparrow$: il bound si allarga (più capacità $\Rightarrow$ rischio di overfitting).

SRM (Structural Risk Minimization): seleziona il modello che minimizza il bound sull'errore vero, ovvero il minimo di $E_{tr}(h) + \text{complexity penalty}$. È la base teorica di SVM (Chapter 8).

Trade-off fondamentale della SLT

$$\underbrace{R(h)}_{\text{Rischio vero}} \leq \underbrace{E_{tr}(h)}_{\text{Errore empirico}} + \underbrace{\Omega(d_{VC}, \ell)}_{\text{Complessità}}$$

Per minimizzare il rischio occorre trovare il giusto compromesso tra fitness ai dati e complessità del modello.

7.5 Sample Complexity

La SLT fornisce anche indicazioni sul numero di esempi necessari per garantire la generalizzazione:

Teorema 7.2: Sample Complexity

Per garantire $R(h) - E_{tr}(h) \leq \varepsilon$ con probabilità almeno $1 - \delta$, il numero di esempi necessario è:

$$\ell \geq \frac{1}{\varepsilon} \left(d_{VC} \ln \frac{2}{\varepsilon} + \ln \frac{2}{\delta} \right) \quad (7.2)$$

Regola empirica: per una buona generalizzazione servono circa $\ell \approx 10 \cdot d_{VC}$ esempi.

7.6 Criteri Analitici per la Selezione del Modello

Oltre a SRM, esistono criteri analitici (approssimati) per la selezione del modello senza hold-out:

- **AIC** (Akaike Information Criterion): penalizza con $2d$ (numero di parametri).
- **BIC** (Bayesian Information Criterion): penalizza con $d \ln \ell$.
- **MDL** (Minimum Description Length): seleziona il modello che permette la descrizione più corta di dati + modello.

Questi criteri valgono per modelli lineari in parametri con errori gaussiani; per NN non lineari la CV rimane la scelta più robusta.

7.7 Limiti Pratici della SLT

- Il bound VC è tipicamente molto lasco (non stretto): dà garanzie conservative non utili in pratica.

- La d_{VC} di reti neurali profonde può essere molto grande (proporzionale al numero di pesi), ma in pratica generalizzano bene — fenomeno non spiegato dalla SLT classica (*double descent*).
- Teorie più recenti (PAC-Bayes, implicit regularization di SGD, benign overfitting) cercano di spiegare il successo del deep learning.

Chapter 8

Support Vector Machines (SVM)

8.1 Introduzione e Motivazione

Le **Support Vector Machines (SVM)**, sviluppate da Vapnik e collaboratori, sono classificatori che **cercano il massimo margine di separazione tra le classi**. Le loro caratteristiche principali sono:

- **Connessione** diretta con la **SLT** (minimizzazione del rischio strutturale).
- **Problema di ottimizzazione convesso**: **il minimo trovato è globale**.
- Capacità di gestire **spazi di feature ad alta dimensionalità** tramite il **kernel trick**.
- **Robustezza** relativa **all'overfitting** grazie al principio del margine.

8.2 SVM a Margine Massimo (**Hard Margin**)

8.2.1 Setup

Classificazione binaria: $y^{(p)} \in \{-1, +1\}$. Il classificatore lineare è:

$$h(\mathbf{x}) = \text{sign}(\mathbf{w}^T \mathbf{x} + b)$$

8.2.2 Il Concetto di Margine

Il **margine** geometrico è la **distanza minima tra il confine di decisione $\{\mathbf{x} : \mathbf{w}^T \mathbf{x} + b = 0\}$ e i punti più vicini di ciascuna classe (i **support vectors**)**.

Per un **classificatore con $\|\mathbf{w}\| = 1$** , la distanza di un punto $\mathbf{x}^{(p)}$ dall'iperpiano è **$|\mathbf{w}^T \mathbf{x}^{(p)} + b|$** . Il **margine totale** (tra i due iperpiani di supporto) è:

$$\rho = \frac{2}{\|\mathbf{w}\|}$$

8.2.3 Problema di Ottimizzazione

L'SVM cerca l'iperpiano che *massimizza* il margine $\rho = 2 / \|\mathbf{w}\|$, ovvero *minimizza* $\|\mathbf{w}\|^2$, soggetto alla corretta classificazione:

$$\begin{aligned} \min_{\mathbf{w}, b} \quad & \frac{1}{2} \|\mathbf{w}\|^2 \\ \text{s.t.} \quad & y^{(p)}(\mathbf{w}^T \mathbf{x}^{(p)} + b) \geq 1, \quad p = 1, \dots, \ell \end{aligned} \quad (8.1)$$

I punti che soddisfano $y^{(p)}(\mathbf{w}^T \mathbf{x}^{(p)} + b) = 1$ si trovano esattamente sul margine e sono detti **support vectors**.

8.2.4 Formulazione Duale

Introducendo i moltiplicatori di Lagrange $\alpha_p \geq 0$ si ottiene il problema duale:

$$\begin{aligned} \max_{\alpha} \quad & \sum_{p=1}^{\ell} \alpha_p - \frac{1}{2} \sum_{p,q} \alpha_p \alpha_q y^{(p)} y^{(q)} \mathbf{x}^{(p)T} \mathbf{x}^{(q)} \\ \text{s.t.} \quad & \alpha_p \geq 0, \quad \sum_{p=1}^{\ell} \alpha_p y^{(p)} = 0 \end{aligned} \quad (8.2)$$

Dalla soluzione duale α^* , i pesi si ricavano come:

$$\mathbf{w}^* = \sum_{p=1}^{\ell} \alpha_p^* y^{(p)} \mathbf{x}^{(p)}$$

Osservazione chiave: solo i support vectors hanno $\alpha_p > 0$; tutti gli altri esempi non influenzano la soluzione.

8.3 Soft Margin SVM

Per dati *non linearmente separabili* o rumorosi, si introduce la **soft margin** con variabili di slack $\xi_p \geq 0$:

$$\begin{aligned} \min_{\mathbf{w}, b, \xi} \quad & \frac{1}{2} \|\mathbf{w}\|^2 + C \sum_{p=1}^{\ell} \xi_p \\ \text{s.t.} \quad & y^{(p)}(\mathbf{w}^T \mathbf{x}^{(p)} + b) \geq 1 - \xi_p \\ & \xi_p \geq 0, \quad p = 1, \dots, \ell \end{aligned} \quad (8.3)$$

dove:

- $\xi_p = 0$: il punto è correttamente classificato con margine ≥ 1 .
- $0 < \xi_p \leq 1$: il punto è sul lato corretto ma dentro al margine.

- $\xi_p > 1$: il punto è sul lato sbagliato (misclassificato).
- $C > 0$: parametro di **regolarizzazione** (iperparametro).

8.3.1 Ruolo di C

- **C grande**: penalizza fortemente gli errori $\Rightarrow$ **margin piccolo**, pochi errori di training, **rischio overfitting**.
- **C piccolo**: tollera più errori $\Rightarrow$ **margin grande**, più **regolarizzazione**.
- Si sceglie C via **cross-validation**.

8.4 Kernel Trick

8.4.1 Feature Map non Lineare

Per **dati non linearmente separabili nello spazio originale**, si può applicare una **trasformazione** $\phi: \mathbb{R}^n \rightarrow \mathcal{F}$ verso uno **spazio di feature $\mathcal{F}$** (possibilmente di dimensione molto alta o infinita) dove la separazione lineare è possibile.

Il **classificatore SVM** nello spazio $\mathcal{F}$:

$$h(\mathbf{x}) = \text{sign} \left(\sum_p \alpha_p y^{(p)} \phi(\mathbf{x}^{(p)})^T \phi(\mathbf{x}) + b \right)$$

8.4.2 Kernel Function

L'idea chiave è che **non serve calcolare $\phi(\mathbf{x})$ esplicitamente**: **basta il kernel**:

$$K(\mathbf{x}, \mathbf{x}') = \phi(\mathbf{x})^T \phi(\mathbf{x}') \quad (8.4)$$

Se K è un kernel valido (simmetrico e semi-definito positivo — **Mercer's theorem**), la formulazione **duale** diventa:

$$\max_{\alpha} \sum_p \alpha_p - \frac{1}{2} \sum_{p,q} \alpha_p \alpha_q y^{(p)} y^{(q)} K(\mathbf{x}^{(p)}, \mathbf{x}^{(q)}) \quad (8.5)$$

8.4.3 Kernel Comuni

Kernel	Formula	Feature space
Lineare	$\mathbf{x}^T \mathbf{x}'$	$\mathbb{R}^n$
Polinomiale	$(\mathbf{x}^T \mathbf{x}' + c)^d$	$O(n^d)$ dimensional
RBF (Gaussiano)	$\exp(-\gamma \ \mathbf{x} - \mathbf{x}'\ ^2)$	∞ dim.
Sigmoid	$\tanh(\kappa \mathbf{x}^T \mathbf{x}' - \delta)$	(non sempre valido)

Kernel RBF: per $\gamma \rightarrow \infty$ avvicina il comportamento al 1-nn; per $\gamma \rightarrow 0$ tende a un classificatore costante. Si seleziona γ via CV.

8.5 SVM per Regressione (SVR)

Le SVM si estendono alla regressione con la ε -insensitive loss:

$$L_\varepsilon(y, h(\mathbf{x})) = \max(0, |y - h(\mathbf{x})| - \varepsilon)$$

Errori minori di ε vengono ignorati (tolleranza). La formulazione prende il nome di Support Vector Regression (SVR).

8.6 Pro e Contro delle SVM

Vantaggi

- **Garanzie teoriche** (SRM): legate alla VC dimension dell'iperpiano.
- Problema convesso: **minimo globale garantito**.
- **Efficace in alta dimensionalità grazie ai kernel**.
- Soluzione **sparsa**: dipende solo dai support vectors.

Svantaggi

- **Scalabilità**: algoritmi standard $O(\ell^2)$ - $O(\ell^3)$; difficile per $\ell > 20000$ (soluzioni moderne usano gradient descent).
- **Scelta del kernel e dei suoi parametri** (non automatica, richiede CV).
- L'**interpretazione probabilistica** dell'output **non è diretta**.
- Per dati molto rumorosi, **C deve essere calibrato** accuratamente.

Nota 8.1: SVM vs. NN

La **differenza** pratica tra SVM e NN tende a **ridursi con dati abbondanti**. Le **NN scalano meglio**, sono più flessibili (architettura), ma sono **più difficili da ottimizzare** (non convexe). Le **SVM hanno garanzie teoriche più solide** (hard margin) **ma le perdono con soft margin e kernel**.

Chapter 9

Bias-Variance Tradeoff e Ensemble Methods

9.1 Decomposizione Bias-Varianza

La decomposizione bias-varianza fornisce un framework formale per comprendere l'errore di generalizzazione. L'idea chiave: un training set è *solo una* delle possibili realizzazioni dei dati. Modelli diversi addestrati su training set diversi produrranno ipotesi diverse.

9.1.1 Setup Formale

Consideriamo il problema di regressione con loss quadratica. Fissato un punto $\mathbf{x}$, l'errore atteso (rispetto alle diverse realizzazioni del training set $\mathcal{D}$ di dimensione ℓ) è:

$$\mathbb{E}_{\mathcal{D}} \left[(y - h_{\mathcal{D}}(\mathbf{x}))^2 \right] \quad (9.1)$$

dove $h_{\mathcal{D}}$ è il modello addestrato su $\mathcal{D}$ e y è il target (potenzialmente affetto da rumore: $y = f(\mathbf{x}) + \varepsilon$).

9.1.2 Decomposizione

Teorema 9.1: Decomposizione Bias-Varianza

L'errore atteso si decompone in tre termini:

$$\mathbb{E}_{\mathcal{D}} \left[(y - h_{\mathcal{D}}(\mathbf{x}))^2 \right] = \underbrace{(\mathbb{E}_{\mathcal{D}}[h_{\mathcal{D}}(\mathbf{x})] - f(\mathbf{x}))^2}_{\text{Bias}^2} + \underbrace{\mathbb{E}_{\mathcal{D}} \left[(h_{\mathcal{D}}(\mathbf{x}) - \mathbb{E}_{\mathcal{D}}[h_{\mathcal{D}}(\mathbf{x})])^2 \right]}_{\text{Varianza}} + \underbrace{\mathbb{E}[\varepsilon^2]}_{\text{Noise}} \quad (9.2)$$

Derivazione: sia $\bar{h}(\mathbf{x}) = \mathbb{E}_{\mathcal{D}}[h_{\mathcal{D}}(\mathbf{x})]$ la predizione media (su tutti i possibili training

set). Usando il lemma $\mathbb{E}[Z^2] = \bar{Z}^2 + \text{Var}[Z]$:

$$\begin{aligned}\mathbb{E}[(y - h)^2] &= \mathbb{E}[(y - \bar{h} + \bar{h} - h)^2] \\ &= (y - \bar{h})^2 + \mathbb{E}[(\bar{h} - h)^2] + 2(y - \bar{h}) \underbrace{\mathbb{E}[\bar{h} - h]}_{=0} \\ &= (f(\mathbf{x}) - \hat{h}(\mathbf{x}))^2 + \text{Var}[h] + \text{Var}[\varepsilon]\end{aligned}$$

9.1.3 Interpretazione

- **Bias**²: misura quanto la predizione media del modello si discosta dalla funzione vera $f(\mathbf{x})$. Cattura l'errore sistematico dovuto alle assunzioni del modello (inductive bias). Alto per modelli troppo semplici.
- **Varianza**: misura quanto la predizione cambia al variare del training set. Alta per modelli molto flessibili che si adattano al rumore.
- **Noise** (rumore irriducibile): varianza del rumore nei dati — non eliminabile. Limite inferiore dell'errore.

Trade-off Bias-Varianza

$$\text{Errore totale} = \text{Bias}^2 + \text{Varianza} + \text{Rumore}$$

Non si può ridurre contemporaneamente bias e varianza aumentando la complessità del modello: ridurre il bias (modelli più complessi) tende ad aumentare la varianza, e viceversa. L'obiettivo è trovare il punto di complessità ottimale.

9.1.4 Esempi

Modello	Bias	Varianza
Modello costante	Alto	Quasi zero
Regressione lineare	Medio	Bassa
K -nn ($K = 1$)	Basso	Alta
K -nn ($K = \ell$)	Alto	Quasi zero
MLP poco profondo	Medio	Media
MLP profondo	Basso	Alta (senza regolarizzazione)

9.2 Ensemble Methods

Gli **ensemble methods** combinano più modelli (*base learners*) per ridurre il bias e/o la varianza. L'intuizione è che diversi modelli faranno errori diversi, e la loro combinazione produce un errore medio inferiore.

9.2.1 Effetto dell'Ensemble sulla Varianza

Siano $h_1, \dots, h_B$ modelli indipendenti con la stessa varianza σ^2 . Il modello ensemble è $\bar{h}(\mathbf{x}) = \frac{1}{B} \sum_{b=1}^B h_b(\mathbf{x})$. Allora:

$$\text{Var}[\bar{h}(\mathbf{x})] = \frac{\sigma^2}{B} \xrightarrow{B \rightarrow \infty} 0$$

Se i modelli non sono indipendenti (correlazione ρ):

$$\text{Var}[\bar{h}] = \rho\sigma^2 + \frac{1-\rho}{B}\sigma^2$$

Conclusione: più modelli e meno correlati $\Rightarrow$ varianza più bassa.

9.2.2 Bagging (Bootstrap Aggregating)

Bagging (Breiman, 1996) riduce la varianza senza aumentare il bias:

1. Genera B campioni bootstrap $\mathcal{D}^{*1}, \dots, \mathcal{D}^{*B}$ (campionamento con ripetizione da $\mathcal{D}$).
2. Addestra un modello h_b su ciascun $\mathcal{D}^{*b}$.
3. Combina le predizioni:
 - Regressione: $\bar{h}(\mathbf{x}) = \frac{1}{B} \sum_b h_b(\mathbf{x})$.
 - Classificazione: majority voting.

Quando è utile? Con modelli ad alta varianza (es. alberi di decisione profondi). Modelli stabili beneficiano poco del bagging.

9.2.3 Random Forest

Random Forest estende il bagging aggiungendo diversità:

- Addestra B alberi di decisione su campioni bootstrap.
- Ad ogni split, considera solo un sottoinsieme casuale di $m \leq n$ feature.
- Riduce la correlazione tra gli alberi rispetto al bagging puro.

9.2.4 Boosting

Il boosting riduce principalmente il bias (non la varianza) addestrando modelli sequenzialmente, dove ogni modello si concentra sugli esempi mal classificati dal precedente.

AdaBoost (Freund & Schapire, 1997):

1. Inizializza **pesi uniformi** $w_p = 1/\ell$.
2. Per $b = 1, \dots, B$:
 - (a) **Addestra** h_b minimizzando l'errore pesato.
 - (b) Calcola $\varepsilon_b = \sum_p w_p \mathbf{1}[h_b(\mathbf{x}^{(p)}) \neq y^{(p)}]$.
 - (c) $\alpha_b = \frac{1}{2} \ln \frac{1-\varepsilon_b}{\varepsilon_b}$.
 - (d) **Aggiorna pesi**: $w_p \leftarrow w_p \exp(-\alpha_b y^{(p)} h_b(\mathbf{x}^{(p)}))$.
3. $H(\mathbf{x}) = \text{sign}(\sum_b \alpha_b h_b(\mathbf{x}))$.

9.2.5 Gradient Boosting

Il **gradient boosting** **generalizza boosting per qualsiasi loss differenziabile**: ogni nuovo modello viene **addestrato sui residui** (pseudo-residui, ovvero i negativi dei gradienti della loss) del modello precedente. **XGBoost, LightGBM** sono implementazioni altamente ottimizzate molto usate.

9.2.6 Stacking

Lo **stacking** **combina le predizioni di diversi modelli** (di tipo diverso) **tramite un meta-learner** (di secondo livello):

- Livello 0: modelli base (es. SVM, RF, NN) producono predizioni su dati OOB.
- Livello 1: un meta-modello (es. regressione logistica) combina le predizioni del livello 0.

9.3 Bias-Varianza per Classificazione

Per la **classificazione** la decomposizione formale è più complessa (loss 0-1 non quadratica), ma l'intuizione rimane valida:

- **Alta varianza**: **il classificatore è sensibile al training set**; piccole perturbazioni cambiano la classificazione.
- **Alto bias**: **il classificatore sbaglia sistematicamente su alcune regioni dello spazio** (confine di decisione troppo semplice).

Chapter 10

Convolutional Neural Networks (CNN)

10.1 Motivazione: Elaborazione di Dati Strutturati

Le **Convolutional Neural Networks** (CNN o ConvNet) sono state progettate specificamente per **dati con struttura spaziale** (immagini) o **temporale** (segnali 1D). Il problema con un MLP standard su immagini:

- Un'immagine 256×256 ha 65536 input — l'**MLP** fully-connected genera un **numero enorme di parametri**.
- L'MLP standard **non sfrutta la *lokalità spaziale*** (feature locali come bordi, angoli) né la ***traslazione invariante*** (una feature può apparire in qualsiasi posizione).

Idea fondamentale: **incorporare la conoscenza a priori** sulla **struttura** dei dati nell'architettura della rete tramite:

1. **Local receptive fields** (connessioni locali).
2. **Weight sharing** (stesso filtro applicato in posizioni diverse).
3. **Pooling** (sub-sampling per invarianza spaziale).

10.2 L'Operazione di Convoluzione

10.2.1 Convoluzione 1D

Per un **segnale 1D $\mathbf{x}$** e un **filtro** (kernel) **$\mathbf{w}$** di dimensione **m** , l'output **o_t** al timestep **t** è:

$$o_t = \sum_{i=1}^m w_i \cdot x_{t+i-1}$$

Il filtro scorre sull'input (sliding window) con uno **stride** s (passo). I pesi w_i sono **condivisi** tra tutte le posizioni (weight sharing).

10.2.2 Convoluzione 2D (per Immagini)

Per un'immagine bidimensionale $\mathbf{I}$ e un kernel $\mathbf{K}$ di dimensione $k \times k$ (es. 3×3):

$$(\mathbf{I} * \mathbf{K})[i, j] = \sum_{m=0}^{k-1} \sum_{n=0}^{k-1} \mathbf{K}[m, n] \cdot \mathbf{I}[i + m, j + n]$$

In forma di cross-correlazione (usata nelle librerie NN come PyTorch/TensorFlow):

$$O[i, j] = \sum_m \sum_n K[m, n] \cdot I[i + m, j + n]$$

Il kernel scorre sull'intera immagine producendo una **feature map** (mappa delle features rilevate in quella posizione).

10.2.3 Padding e Stride

- **Padding**: aggiunge **bordi di zeri** attorno all'immagine per preservare le **dimensioni spaziali**. Con padding p e kernel k , output: $n - k + 2p + 1$.
 - *Valid*: nessun padding, output ridotto.
 - *Same*: padding tale che output abbia stesse dimensioni dell'input.
- **Stride** s : **passo dello spostamento del kernel**. Output: $\lfloor (n - k) / s \rfloor + 1$. Stride > 1 riduce la dimensione spaziale.

10.3 Architettura CNN

Una CNN tipica alterna **tre tipi di layer**:

10.3.1 Convolutional Layer

- Contiene **F filtri** (kernel) appresi, ciascuno di dimensione **$k \times k \times C$** (con C canali in ingresso).
- Ogni filtro produce una **feature map**: F filtri producono F feature map.
- Numero di parametri per layer: $F \times k \times k \times C + F$ (bias).
- **Weight sharing**: lo **stesso filtro è applicato a tutte le posizioni** $\Rightarrow$ drastica riduzione dei parametri rispetto a fully-connected.

10.3.2 Pooling Layer

Il pooling **riduce le dimensioni spaziali della feature map, aumentando l'invarianza alla traslazione:**

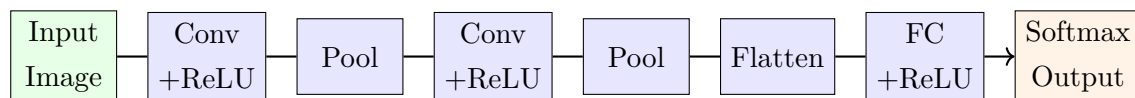
- **Max pooling:** prende il massimo in una finestra $k \times k$. Preserva le attivazioni più forti.
- **Average pooling:** prende la media nella finestra.
- **Global Average Pooling (GAP):** riduce ogni feature map a un singolo scalare (media globale). Usato nelle architetture moderne.

Il pooling non ha parametri appresi.

10.3.3 Fully-Connected Layer

Dopo i layer convoluzionali e di pooling, le **feature map vengono appiattite (*flatten*) e passate a uno o più layer fully-connected** per la classificazione finale.

10.3.4 Schema Tipico (LeNet-like)



10.4 Proprietà delle CNN

- **Local connectivity:** ogni neurone è connesso solo a una regione locale dell'input (receptive field). Rispecchia la struttura del sistema visivo biologico.
- **Translation invariance:** grazie al **weight sharing**, la **CNN rileva la stessa feature indipendentemente dalla sua posizione** nell'immagine.
- **Feature hierarchy:** i layer più profondi combinano **feature locali in feature sempre più astratte**:
 - Layer 1: bordi, angoli, frequenze spaziali.
 - Layer 2: texture, parti di oggetti.
 - Layer profondi: concetti astratti (facce, oggetti).
- **Riduzione dei parametri:** peso condiviso **riduce** drasticamente il **numero** di **parametri** rispetto all'MLP fully-connected.

10.5 Architetture CNN Importanti

Architettura	Anno	Contributi chiave
LeNet-5	1998	Prima CNN moderna (LeCun et al.)
AlexNet	2012	GPU training, ReLU, Dropout, top-5 ImageNet
VGGNet	2014	Kernel 3×3 , profondità
GoogLeNet	2014	Inception module, 1×1 conv
ResNet	2015	Skip connections (residual learning)
DenseNet	2017	Dense connections
EfficientNet	2019	Scaling composto

10.5.1 Residual Connections (ResNet)

Il **problema del vanishing gradient** nelle reti molto profonde viene risolto da He et al. (2015) con le **skip connections**:

$$F(\mathbf{x}) + \mathbf{x}$$

dove $F(\mathbf{x})$ è l'output del blocco e $\mathbf{x}$ è l'input che bypassa il blocco. La rete apprende il **residuo** rispetto all'identità:

$$h(\mathbf{x}) = F(\mathbf{x}) + \mathbf{x}$$

Il gradiente può fluire direttamente attraverso le **skip connections**, consentendo l'addestramento di reti con centinaia di layer.

10.6 Training delle CNN

Il training avviene con la **backpropagation estesa all'operazione di convoluzione**. La backprop attraverso un layer convoluzionale **produce**:

- Il gradiente rispetto ai **pesi del filtro**: media delle correlazioni locali tra input e gradiente dell'output.
- Il gradiente rispetto all'**input**: convoluzione del gradiente con il filtro ribaltato (trasposizione).

Data Augmentation: per aumentare artificialmente il training set, si applicano **trasformazioni alle immagini** (rotazione, flip, crop, scaling, color jitter). **Riduce l'overfitting e aumenta l'invarianza**.

Transfer Learning: **pre-addestrare la rete su un dataset grande** (es. ImageNet) e **fine-tunare sullo specifico task**. Particolarmente efficace quando i dati del task target sono limitati.

Chapter 11

Deep Neural Networks

11.1 Cos'è il Deep Learning?

Definizione 11.1: Deep Learning

Il **Deep Learning** consiste nell'uso di modelli computazionali composti da **molti strati** di elaborazione non lineare per apprendere **rappresentazioni dei dati con livelli crescenti di astrazione**. (LeCun, Bengio & Hinton, *Nature*, 2015)

11.1.1 Shallow vs. Deep Models

- **Shallow** model: **NN con pochi layer** (es. MLP con 1 hidden layer). Per approssimare funzioni complesse richiede un numero esponenzialmente grande di neuroni nel singolo hidden layer.
- **Deep** model: **MLP con molti layer (≥ 3)**. Può rappresentare funzioni complesse con **meno parametri** totali sfruttando la **composizione gerarchica**.

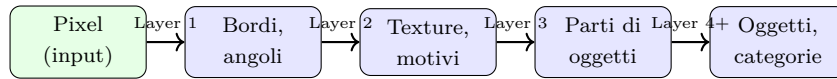
11.1.2 Fattori Abilitanti del Deep Learning (dal 2010)

1. **Big data**: dataset di dimensioni enormi disponibili (milioni di immagini, testi, ecc.).
2. **Hardware**: GPU e cluster HPC che permettono il training di reti profonde in tempi ragionevoli.
3. **Nuove tecniche**: ReLU, Dropout, Batch Normalization, inizializzazione avanzata, ottimizzatori adattativi.

11.2 Rappresentazione Gerarchica

Il concetto chiave del Deep Learning è la **hierarchical feature learning**: ogni layer apprende rappresentazioni sempre più astratte dei dati.

Esempio con immagini:



Distributed representation: le informazioni sono codificate in modo distribuito su molti neuroni, non localizzato in singole unità — questo permette una rappresentazione esponenzialmente efficiente.

11.3 Perché i Layer Multipli Sono Utili?

Teorema 11.1: Efficienza della Profondità

Alcune funzioni (es. funzioni che calcolano la parità di n bit) richiedono un numero esponenziale di nodi in un MLP a singolo hidden layer, ma sono rappresentabili con un numero polinomiale in reti profonde.

La composizione di funzioni semplici permette di decomporre problemi complessi in sottoproblemi più semplici — una forma di "divide et impera" computazionale.

11.4 Tecniche per il Training di Reti Profonde

11.4.1 Vanishing/Exploding Gradient

In reti profonde con backpropagation, il gradiente può:

- **Vanire** (vanishing gradient): il gradiente diminuisce esponenzialmente nei layer profondi $\Rightarrow$ pesi quasi invarianti, apprendimento bloccato (problematico con sigmoide/tanh).
- **Esplosione** (exploding gradient): il gradiente cresce esponenzialmente $\Rightarrow$ aggiornamenti instabili.

11.4.2 Batch Normalization (BatchNorm)

Batch Normalization (Ioffe & Szegedy, 2015): normalizza le attivazioni di ogni layer per ogni mini-batch, poi scala e trasla con parametri appresi:

$$\hat{x}_i = \frac{x_i - \mu_B}{\sqrt{\sigma_B^2 + \varepsilon}}, \quad y_i = \gamma \hat{x}_i + \beta \quad (11.1)$$

dove μ_B, σ_B^2 sono media e varianza del mini-batch, γ, β sono parametri appresi.

Vantaggi:

- Riduce il problema del *internal covariate shift*.
- Permette learning rate più alti.
- Effetto regolarizzante (riduce la necessità di dropout).
- Accelera la convergenza.

11.4.3 Dropout

Dropout (Srivastava et al., 2014): durante il training, ogni neurone viene disattivato (output=0) con probabilità p ad ogni passo:

- In training: $o_i \leftarrow o_i \cdot \text{Bernoulli}(1 - p)/1$ (oppure con scaling: $o_i/(1 - p)$ per mantenere la scala).
- In inference: tutti i neuroni attivi, pesi scalati per $(1 - p)$.

Interpretazione: equivale ad addestrare un ensemble di 2^N sottoreti (con N neuroni) e mediane le predizioni. Forte regolarizzazione.

11.4.4 Ottimizzatori Adattativi

Ottimizzatori che adattano il learning rate per ciascun peso:

Adam (Kingma & Ba, 2014) — il più popolare:

$$m_t = \beta_1 m_{t-1} + (1 - \beta_1) g_t \quad (1^\circ \text{ momento}) \quad (11.2)$$

$$v_t = \beta_2 v_{t-1} + (1 - \beta_2) g_t^2 \quad (2^\circ \text{ momento}) \quad (11.3)$$

$$w_t = w_{t-1} - \frac{\eta}{\sqrt{\hat{v}_t} + \varepsilon} \hat{m}_t \quad (11.4)$$

dove $g_t = \nabla_{w_t} E$, $\hat{m}_t, \hat{v}_t$ sono le versioni corrette per il bias. Tipicamente $\beta_1 = 0.9$, $\beta_2 = 0.999$, $\varepsilon = 10^{-8}$.

Ottimizzatore	Adattivo	Caratteristiche
SGD	No	Semplice, robusto
SGD + Momentum	No	Smorzamento oscillazioni
AdaGrad	Sì	LR decrescente (bene per sparsi)
RMSProp	Sì	Come AdaGrad ma media mobile
Adam	Sì	Momentum + RMSProp, più usato

11.4.5 Pre-Training (Layer-wise)

Tecnica storica per inizializzare reti profonde (Hinton et al., 2006):

1. Pre-addestrare ciascun layer in modo non supervisionato (es. Restricted Boltzmann Machine, autoencoder).
2. Usare i pesi pre-addestrati come inizializzazione per il fine-tuning supervisionato con backpropagation.

Con ReLU, BatchNorm e ottimizzatori moderni, il pre-training è meno necessario.

11.5 Architetture Deep: Overview

Architettura	Task principale
MLP profondo	Dati tabulari, classificazione generica
CNN profonda	Computer vision
RNN/LSTM	Sequenze temporali, NLP
Transformer	NLP, Vision (ViT)
GAN	Generazione di immagini
VAE	Generazione, rappresentazioni latenti
GNN	Grafi, strutture relazionali

11.6 Transfer Learning e Fine-Tuning

Definizione 11.2: Transfer Learning

Il **transfer learning** consiste nell'utilizzare la conoscenza acquisita addestrandosi su un task/dataset sorgente per migliorare le performance su un task/dataset target diverso.

Strategia tipica (per CNN e NLP):

1. Pre-addestrare una rete su un dataset grande (es. ImageNet per visione, Wikipedia per NLP).
2. Sostituire l'output layer con uno adatto al nuovo task.
3. **Fine-tuning**: addestrare l'intera rete (o solo gli ultimi layer) sul dataset target con learning rate basso.

Feature extraction: usare i layer convoluzionali pre-addestrati come estrattori di feature fissi (senza fine-tuning), addestrando solo il classificatore.

Chapter 12

Randomized Neural Networks

12.1 Introduzione agli Approcci Randomizzati

Gli **approcci randomizzati** in Machine Learning introducono **casualità** come parte integrante della **strategia** costruttiva del modello. La randomizzazione può:

- **Migliorare le performance** predittive (ensemble, data augmentation).
- **Alleviare difficoltà computazionali** del training classico.
- Costituire il **core design** del modello stesso.

12.1.1 Randomness in ML: Livelli di Applicazione

- **Data processing**: data augmentation, subsampling.
- **Learning e selezione degli iperparametri**: SGD stocastico, random search per hyperparameter optimization, dropout.
- **Architettura del modello**: pesi fissi casuali, reservoir computing.

12.2 Extreme Learning Machine (ELM)

Extreme Learning Machine (Huang et al., 2004) è un **MLP** con un **singolo hidden layer** dove i pesi del **hidden layer non vengono addestrati**:

Algoritmo 5: Extreme Learning Machine

1. Scegli m (numero di unità hidden)
2. **Inizializza casualmente** i pesi del hidden layer: $\mathbf{W}^{(1)} \sim \mathcal{U}[-a, a]$
3. Calcola le **attivazioni hidden** (non lineari): $\mathbf{H} = f(\mathbf{X}\mathbf{W}^{(1)T}) \in \mathbb{R}^{\ell \times m}$

4. Addestra i **pesi di output** con Least Squares (soluzione analitica): $\mathbf{W}^{(2)} = (\mathbf{H}^T \mathbf{H})^{-1} \mathbf{H}^T \mathbf{Y}$

12.2.1 Analisi

- **Training istantaneo**: risolve un sistema lineare (una sola operazione, nessuna iterazione).
- **Universalità**: con sufficienti neuroni nascosti, un ELM con pesi random può **approssimare qualsiasi funzione** (teorema).
- **Varianza**: l'inizializzazione casuale introduce **varianza** nei risultati — necessario mediare su più realizzazioni (ensemble di ELM).
- **Iperparametri**: numero di unità hidden m e distribuzione dei pesi random (varianza).

ELM come kernel method

$\mathbf{H}(\mathbf{H}^T \mathbf{H})^{-1} \mathbf{H}^T = \mathbf{K}$ dove $K_{pq} = f(\mathbf{w}^T \mathbf{x}^{(p)}) \cdot f(\mathbf{w}^T \mathbf{x}^{(q)})$ è un **kernel implicito** definito dai pesi random. **L'ELM è quindi equivalente a un kernel method con kernel randomizzato.**

12.3 Random Kitchen Sinks e Random Features

Random Features (Rahimi & Recht, 2007): approssimano kernel shift-invariant (es. RBF) tramite proiezioni casuali:

$$K(\mathbf{x}, \mathbf{x}') = \mathbb{E}_{\mathbf{w} \sim p(\mathbf{w})} [z_{\mathbf{w}}(\mathbf{x}) z_{\mathbf{w}}(\mathbf{x}')] \approx \frac{1}{D} \sum_{d=1}^D z_{\mathbf{w}_d}(\mathbf{x}) z_{\mathbf{w}_d}(\mathbf{x}') \quad (12.1)$$

dove $z_{\mathbf{w}}(\mathbf{x}) = \cos(\mathbf{w}^T \mathbf{x} + b)$ con $\mathbf{w}$ campionato da $p(\mathbf{w}) = \mathcal{F}[K]$ (trasformata di Fourier del kernel).

Questo permette di **scalare metodi kernel a grandi dataset con complessità $O(\ell D)$ invece di $O(\ell^2)$.**

12.4 Reservoir Computing ed Echo State Network

Il **Reservoir Computing** è un paradigma per l'elaborazione di sequenze temporali con reti neurali ricorrenti parzialmente random:

- **Reservoir**: una rete ricorrente con molte connessioni *fisse e casuali* — agisce come un proiettore in uno spazio ricco di feature temporali.
- **Readout layer**: solo l'output layer viene addestrato (soluzione lineare).

Echo State Network (ESN): implementazione specifica in cui il reservoir è una RNN sparsa con pesi random e il readout è un layer lineare. Il training si riduce alla sola regressione lineare — estremamente efficiente.

Condizione di echo state: lo spettro della matrice di peso del reservoir deve avere **raggio spettrale** $\rho(\mathbf{W}^{rec}) < 1$ per garantire che la rete "dimentichi" gli stati precedenti (condizione di stabilità).

12.5 Random Forests come Randomized ML

Le **Random Forests** (già discusse nel Chapter 9) sono un esempio di approccio randomizzato per modelli ad albero:

- Campionamento bootstrap del training set.
- Selezione casuale del sottoinsieme di feature ad ogni split.

La randomizzazione introduce diversità tra gli alberi, riducendo la correlazione e quindi la varianza dell'ensemble.

12.6 Dropout come Randomizzazione

Il **Dropout** (già visto nel Chapter 11) può essere visto come una forma di randomizzazione dell'architettura:

- Ad ogni forward pass, si addestra una diversa sotto-rete.
- L'ensemble implicito di 2^N sottoreti riduce la varianza.

MC Dropout (Gal & Ghahramani, 2016): mantenere il dropout anche in inference e usare campioni multipli per stimare l'incertezza del modello (approximate Bayesian inference).

12.7 Vantaggi e Limiti degli Approcci Randomizzati

Vantaggi

- **Efficienza computazionale**: training molto più rapido (ELM, ESN).
- **Diversità e ensemble**: riduzione della varianza.
- **Scalabilità**: random features per kernel methods su grandi dataset.

Limiti

- Alta varianza nelle performance (dipendenza dall'inizializzazione casuale).
- Meno adatti quando la struttura dei dati richiede rappresentazioni precise.
- I pesi random non ottimizzati limitano la capacità del modello rispetto al training completo.

Chapter 13

Unsupervised Learning con Reti Neurali

13.1 Unsupervised Learning: Introduzione

Nell'**unsupervised learning** **non sono disponibili etichette**: il training set è $\mathcal{D} = \{\mathbf{x}^{(p)}\}_{p=1}^{\ell}$.

Obiettivi principali:

- **Clustering**: trovare raggruppamenti naturali nei dati.
- **Dimensionality reduction**: proiettare i dati in uno spazio di dimensione inferiore preservando la struttura.
- **Density estimation**: modellare la **distribuzione** dei **dati** $P(\mathbf{x})$.
- **Feature learning**: apprendere rappresentazioni utili per task successivi.

Utilità: i **dati non etichettati** sono molto più economici da raccogliere degli etichettati — fondamentale per applicazioni reali su larga scala.

13.2 Clustering e Vector Quantization

13.2.1 Il Problema del Clustering

Il clustering partiziona i dati in **K cluster** tali che:

- Dati nello **stesso cluster** siano **simili** (distanza piccola).
- Dati in **cluster diversi** siano **dissimili** (distanza grande).

13.2.2 Vector Quantization (VQ)

La **vector quantization** descrive una **varietà di dati** tramite un **insieme finito di prototipi** (centroidi) $\{\mathbf{w}_1, \dots, \mathbf{w}_K\}$.

Per ogni dato $\mathbf{x}$, il **prototipo** vincente è:

$$i^*(\mathbf{x}) = \arg \min_{i \in \{1, \dots, K\}} d(\mathbf{x}, \mathbf{w}_i)$$

Le **regioni di Voronoi** $V_i = \{\mathbf{x} : \mathbf{w}_i = \arg \min_j d(\mathbf{x}, \mathbf{w}_j)\}$ definiscono la partizione dello spazio.

Loss (distorsione totale):

$$E = \sum_{p=1}^{\ell} d(\mathbf{x}^{(p)}, \mathbf{w}_{i^*(\mathbf{x}^{(p)})})$$

Il problema di trovare i prototipi ottimali è NP-completo in generale.

13.2.3 K-Means

K-Means è il metodo più semplice per VQ:

Algoritmo 6: K-Means

1. Inizializza **K centroidi** $\{\mathbf{w}_k\}$ (es. casualmente da $\mathcal{D}$)
2. **repeat**
3. **Assegnazione:** $z^{(p)} = \arg \min_k d(\mathbf{x}^{(p)}, \mathbf{w}_k)$
4. **Aggiornamento:** $\mathbf{w}_k = \frac{1}{|C_k|} \sum_{p: z^{(p)}=k} \mathbf{x}^{(p)}$
- until** **convergenza** (assegnazioni non cambiano)

Converge **sempre a un minimo locale**. Sensibile all'inizializzazione (si usa **K-Means++** per una migliore inizializzazione).

13.3 Self-Organizing Map (SOM)

La **Self-Organizing Map** (SOM), o mappa di Kohonen (Kohonen, 1982), è una rete neurale non supervisionata che combina VQ con la preservazione della topologia.

13.3.1 Architettura

- **Input layer:** **n neuroni** (uno per feature).

- **Competitive layer (mappa)**: griglia di $K = m_1 \times m_2$ neuroni (solitamente 2D). Ogni neurone i ha un **prototipo** $\mathbf{w}_i \in \mathbb{R}^n$.

13.3.2 Algoritmo di **Apprendimento**

Algoritmo 7: SOM Learning

1. Inizializza i prototipi $\mathbf{w}_i$ (casualmente)
2. **for** $t = 1, 2, \dots, T$ **do**
3. Presenta un **input** $\mathbf{x}^{(p)}$ (casuale da $\mathcal{D}$)
4. **Competitive step**: $i^* = \arg \min_i \|\mathbf{x}^{(p)} - \mathbf{w}_i\|$ (Best Matching Unit)
5. **Update**: $\mathbf{w}_i \leftarrow \mathbf{w}_i + \eta(t) \cdot h(i, i^*, t) \cdot (\mathbf{x}^{(p)} - \mathbf{w}_i)$
- end for**

dove:

- $h(i, i^*, t) = \exp\left(-\frac{d_{map}(i, i^*)^2}{2\sigma^2(t)}\right)$ è la **funzione di vicinato** (Gaussian neighborhood) sulla mappa 2D.
- $d_{map}(i, i^*)$ è la **distanza** sulla mappa tra il neurone i e il vincitore i^* .
- $\eta(t)$ e $\sigma(t)$ decrescono nel tempo (raffreddamento).

13.3.3 Proprietà della SOM

- **Topological ordering**: neuroni vicini sulla mappa rispondono a input simili. La mappa preserva la struttura topologica dello spazio degli input.
- **Adaptive resolution**: la densità dei prototipi si adatta alla distribuzione dei dati (più prototipi nelle regioni dense).
- **Visualizzazione**: la mappa 2D permette di visualizzare strutture in spazi ad alta dimensionalità.

13.4 Autoencoder

Un **autoencoder** è una rete neurale addestrata per **ricostruire il suo input**:

$$h(\mathbf{x}) \approx \mathbf{x}$$

Architettura: encoder $\mathbf{z} = f_{\theta}(\mathbf{x})$ (compressione) + decoder $\hat{\mathbf{x}} = g_{\phi}(\mathbf{z})$ (ricostruzione), con **bottleneck** $\mathbf{z} \in \mathbb{R}^d$ con $d \ll n$.

Loss: $\mathcal{L} = \|\mathbf{x} - g_{\phi}(f_{\theta}(\mathbf{x}))\|^2$

Applicazioni: riduzione della dimensionalità, denoising, pretraining.

13.4.1 Variational Autoencoder (VAE)

Estende l'autoencoder a un modello **generativo**: l'encoder produce una distribuzione $q(\mathbf{z}|\mathbf{x}) = \mathcal{N}(\boldsymbol{\mu}, \boldsymbol{\sigma}^2)$ e la loss include un termine KL per regolarizzare la distribuzione latente:

$$\mathcal{L} = \mathbb{E}_q[\log p(\mathbf{x}|\mathbf{z})] - KL(q(\mathbf{z}|\mathbf{x})||p(\mathbf{z}))$$

Chapter 14

Recurrent Neural Networks (RNN)

14.1 Dati Sequenziali e la Necessità di Memoria

Le reti feedforward standard elaborano ogni input indipendentemente, senza memoria degli input precedenti. Molti problemi reali, però, richiedono di considerare la *storia degli input*:

- **NLP**: la comprensione di una parola dipende dal contesto (parole precedenti).
- **Time series**: predire il prossimo valore di una serie temporale.
- **Speech recognition**: riconoscimento del parlato.
- **Bioinformatica**: analisi di sequenze di aminoacidi.

14.1.1 Approcci per la *Memoria* nelle NN

1. **Finite memory** (TDNN): *sliding window sull'input* — finestra fissa di input passati. Semplice ma limitata capacità di memoria.
2. **Infinite memory** (RNN): *connessioni ricorrenti* — lo stato nascosto $\mathbf{h}_t$ cattura informazioni su tutti gli input passati.

14.2 Recurrent Neural Network (Simple RNN)

14.2.1 Architettura

Una RNN standard (*Elman network*, 1990) aggiunge *connessioni ricorrenti al hidden layer*:

$$\mathbf{h}_t = f(\mathbf{W}_{hx}\mathbf{x}_t + \mathbf{W}_{hh}\mathbf{h}_{t-1} + \mathbf{b}_h) \quad (14.1)$$

$$\mathbf{o}_t = g(\mathbf{W}_{oh}\mathbf{h}_t + \mathbf{b}_o) \quad (14.2)$$

dove:

- $\mathbf{x}_t \in \mathbb{R}^n$: input al timestep t .
- $\mathbf{h}_t \in \mathbb{R}^m$: stato nascosto — la "memoria" della rete.
- $\mathbf{o}_t$: output al timestep t .
- $\mathbf{W}_{hx}, \mathbf{W}_{hh}, \mathbf{W}_{oh}$: matrici di pesi (condivise nel tempo).
- $\mathbf{h}_0 = \mathbf{0}$ (stato iniziale).

Weight Sharing Temporale

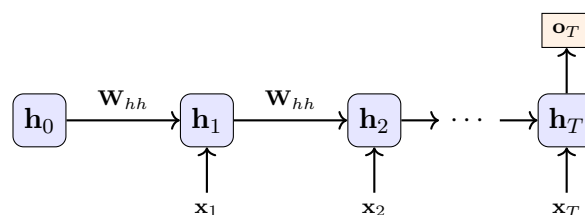
I pesi $\mathbf{W}_{hx}, \mathbf{W}_{hh}, \mathbf{W}_{oh}$ sono **condivisi** tra tutti i timestep — la rete applica la stessa trasformazione ad ogni passo. Questo riduce il numero di parametri ma richiede che la stessa funzione sia adatta a tutti i passi temporali.

14.2.2 Tipi di Task per RNN

- **Sequence classification**: output solo alla fine ($\mathbf{o}_T$). Es. classificazione del sentiment.
- **IO-transduction**: output ad ogni passo. Es. POS tagging.
- **Next-step prediction**: predire $\mathbf{x}_{t+1}$ dall'input $\mathbf{x}_1, \dots, \mathbf{x}_t$. Base dei modelli linguistici autoregressivi.
- **Sequence generation**: generare sequenze da un seme (es. generazione di testo).

14.3 Backpropagation Through Time (BPTT)

Il **training** delle RNN usa **Backpropagation Through Time** (BPTT): la RNN viene "srotolata" nel tempo (unrolling) e la backpropagation viene applicata alla rete srotolata.



Problema: il gradiente deve essere propagato per T passi temporali. Se T è grande:

- **Vanishing gradient**: il **gradiente decade esponenzialmente** $\Rightarrow$ la rete non riesce ad apprendere dipendenze a lungo termine.
- **Exploding gradient**: il gradiente **cresce esponenzialmente** (mitigato con gradient clipping).

Truncated BPTT: **limitare la backpropagation a k passi temporali** per efficienza computazionale.

14.4 Long Short-Term Memory (LSTM)

L'**LSTM** (Hochreiter & Schmidhuber, 1997) **risolve il problema del vanishing gradient introducendo una cell state $\mathbf{c}_t$ e tre gate**:

$$\text{Forget gate: } \mathbf{f}_t = \sigma(\mathbf{W}_f[\mathbf{h}_{t-1}; \mathbf{x}_t] + \mathbf{b}_f) \quad (14.3)$$

$$\text{Input gate: } \mathbf{i}_t = \sigma(\mathbf{W}_i[\mathbf{h}_{t-1}; \mathbf{x}_t] + \mathbf{b}_i) \quad (14.4)$$

$$\text{Candidate: } \tilde{\mathbf{c}}_t = \tanh(\mathbf{W}_c[\mathbf{h}_{t-1}; \mathbf{x}_t] + \mathbf{b}_c) \quad (14.5)$$

$$\text{Cell state: } \mathbf{c}_t = \mathbf{f}_t \odot \mathbf{c}_{t-1} + \mathbf{i}_t \odot \tilde{\mathbf{c}}_t \quad (14.6)$$

$$\text{Output gate: } \mathbf{o}_t = \sigma(\mathbf{W}_o[\mathbf{h}_{t-1}; \mathbf{x}_t] + \mathbf{b}_o) \quad (14.7)$$

$$\text{Hidden state: } \mathbf{h}_t = \mathbf{o}_t \odot \tanh(\mathbf{c}_t) \quad (14.8)$$

dove $\odot$ è il prodotto elemento per elemento (Hadamard).

Meccanismo:

- **Forget gate $\mathbf{f}_t$** : decide **quale parte dello stato precedente $\mathbf{c}_{t-1}$ dimenticare** (valori vicini a 0 = dimentica, 1 = mantieni).
- **Input gate $\mathbf{i}_t$** : controlla **quali nuove informazioni aggiungere**.
- **Cell state $\mathbf{c}_t$** : la **memoria a lungo termine**.
- **Output gate $\mathbf{o}_t$** : controlla **quale parte dello stato è usata per l'output**.

La cell state scorre quasi indisturbata nel tempo (via i gate di forget) permettendo un flusso di gradiente senza vanishing per dipendenze a lungo termine.

14.4.1 Gated Recurrent Unit (GRU)

Il **GRU** (Cho et al., 2014) è una versione semplificata dell'LSTM con **solamente due gate**:

$$\mathbf{r}_t = \sigma(\mathbf{W}_r[\mathbf{h}_{t-1}; \mathbf{x}_t]) \quad (\text{reset gate}) \quad (14.9)$$

$$\mathbf{z}_t = \sigma(\mathbf{W}_z[\mathbf{h}_{t-1}; \mathbf{x}_t]) \quad (\text{update gate}) \quad (14.10)$$

$$\tilde{\mathbf{h}}_t = \tanh(\mathbf{W}[\mathbf{r}_t \odot \mathbf{h}_{t-1}; \mathbf{x}_t]) \quad (14.11)$$

$$\mathbf{h}_t = (1 - \mathbf{z}_t) \odot \mathbf{h}_{t-1} + \mathbf{z}_t \odot \tilde{\mathbf{h}}_t \quad (14.12)$$

Performance comparabile all'LSTM ma con meno parametri.

14.5 Architetture RNN Avanzate

14.5.1 Bidirectional RNN

Processa la sequenza in **entrambe le direzioni** (forward e backward):

$$\mathbf{h}_t = [\vec{\mathbf{h}}_t; \overleftarrow{\mathbf{h}}_t]$$

Utile quando il **contesto futuro è disponibile** (es. traduzione, analisi di testo).

14.5.2 Stacked (Multi-layer) RNN

Più layer RNN impilati, con l'output del layer l come input del layer $l + 1$. Aumenta la capacità di rappresentazione ma è più difficile da addestrare.

14.5.3 Seq2Seq e Attention

Encoder-Decoder (Seq2Seq): un **encoder RNN** processa la sequenza di input e produce un **vettore di contesto $\mathbf{c}$** ; un **decoder RNN** genera la sequenza di output condizionata su **$\mathbf{c}$** .

Attention mechanism: invece di comprimere tutta la sequenza di input in un singolo vettore, **l'attention permette al decoder di accedere selettivamente a tutte le hidden state dell'encoder ad ogni passo:**

$$\alpha_{t,s} = \text{softmax}(e_{t,s}), \quad \mathbf{c}_t = \sum_s \alpha_{t,s} \mathbf{h}_s^{enc}$$

L'attention ha portato ai modelli Transformer (Vaswani et al., 2017) — la base di GPT, BERT, e tutti i Large Language Models moderni.

14.6 RNN vs. Transformers

	RNN/LSTM	Transformer
Parallelismo	Sequenziale	Parallelo
Memoria	Implicita (stato)	Attention esplicita
Dipendenze lunghe	Difficili	Facili (con attention)
Complessità	$O(T \cdot d^2)$	$O(T^2 \cdot d)$
Applicazioni	Streaming, control	NLP, visione

Chapter 15

Learning in Structured Domains e Graph Neural Networks

15.1 Motivazione: Oltre i Dati Piatti

Finora abbiamo considerato dati come vettori $\mathbf{x} \in \mathbb{R}^n$. Molti problemi reali hanno però **struttura relazionale**:

- **Molecole**: atomi = nodi, legami = archi (drug design).
- **Social networks**: utenti = nodi, relazioni = archi.
- **Knowledge graphs**: entità = nodi, relazioni = archi.
- **Proteine**: sequenza/struttura 3D.
- **Analisi del traffico**: strade = grafi.

Dominio di input	Modello	Esempio
Vettori (flat)	MLP, SVM	Classificazione classica
Sequenze	RNN, LSTM	NLP, time series
Immagini	CNN	Computer vision
Grafi/Alberi	GNN, DGN	Chimica, social networks

15.2 Reti Neurali per Strutture: Cenni Storici

L'approccio delle reti neurali per dati strutturati ha origine italiana:

- **Anni '90, Pisa**: Sperduti, Starita, Micheli, Frasconi, Gori — prime reti neurali ricorsive per alberi e grafi orientati aciclici (DPAG).

- **2005–2009**: estensione a grafi generali — GNN (Scarselli et al., 2009), NN4G (Micheli, 2009), GraphESN (Gallicchio & Micheli, 2010).
- **2015 in poi**: esplosione mondiale della Graph Neural Network research — GCN (Kipf & Welling, 2017), GraphSAGE, GAT, etc.

15.3 Il Framework Deep Graph Networks (DGN)

15.3.1 Rappresentazione del Grafo

Un **grafo etichettato** $G = (V, E, \ell)$ è definito da:

- V : insieme di **nodi** (vertici).
- $E \subseteq V \times V$: insieme di **archi** (edges).
- $\ell : V \rightarrow \mathbb{R}^n$: funzione di **etichetta** dei nodi (feature vettoriali).
- $A \in \{0, 1\}^{|V| \times |V|}$: **matrice di adiacenza**.

Task su grafi:

- **Node-level**: **classificazione/regressione** per ogni nodo.
- **Edge-level**: **predizione di link**.
- **Graph-level**: **classificazione/regressione per l'intero grafo**.

15.3.2 Il Principio del Neighborhood Aggregation

Le GNN apprendono **rappresentazioni (embedding)** dei **nodi** aggregando le informazioni **dei vicini** — il principio di **message passing**:

$$\mathbf{h}_v^{(k)} = \text{UPDATE}^{(k)}\left(\mathbf{h}_v^{(k-1)}, \text{AGGREGATE}^{(k)}\left(\{\mathbf{h}_u^{(k-1)} : u \in \mathcal{N}(v)\}\right)\right) \quad (15.1)$$

dove $\mathcal{N}(v)$ è il **vicinato** del nodo v , k è il numero del layer GNN, e $\mathbf{h}_v^{(0)} = \ell(v)$ (feature iniziali del nodo).

Dopo K layer, $\mathbf{h}_v^{(K)}$ incorpora informazioni da tutti i nodi entro distanza K da v — il suo **K -hop neighborhood**.

15.4 Graph Convolutional Network (GCN)

GCN (Kipf & Welling, 2017): la variante più semplice e popolare. La **propagazione** di un layer GCN è:

$$\mathbf{H}^{(k)} = \sigma(\tilde{\mathbf{D}}^{-1/2} \tilde{\mathbf{A}} \tilde{\mathbf{D}}^{-1/2} \mathbf{H}^{(k-1)} \mathbf{W}^{(k)}) \quad (15.2)$$

dove:

- $\tilde{\mathbf{A}} = \mathbf{A} + \mathbf{I}$: matrice di **adiacenza** con self-loop.
- $\tilde{\mathbf{D}}$: matrice **diagonale** dei **gradi** di $\tilde{\mathbf{A}}$.
- $\mathbf{W}^{(k)}$: matrice di **pesi** appresi del layer k .
- $\mathbf{H}^{(0)} = \mathbf{X}$ (feature iniziali dei nodi).

Equivale a una media pesata (normalizzata) delle feature del vicinato.

15.4.1 Graph Neural Network (Scarselli et al., 2009)

Il **GNN originale** usa una formulazione basata su **punto fisso**: ogni nodo v ha uno **stato $\mathbf{x}_v$** che viene **aggiornato iterativamente finché non converge**:

$$\mathbf{x}_v = F_w(\mathbf{1}_v, \sum_{u \in \mathcal{N}(v)} F_w(\mathbf{1}_u, \mathbf{x}_u, \mathbf{1}_{v,u}))$$

La convergenza è **garantita** con condizioni sul contraction mapping.

15.5 Graph Attention Networks (GAT)

Le **GAT** (Veličković et al., 2018) estendono la GCN con un **meccanismo di attention**: **i vicini contribuiscono con pesi diversi appresi**:

$$\alpha_{vu} = \frac{\exp(\text{LeakyReLU}(\mathbf{a}^T [\mathbf{W}\mathbf{h}_v \parallel \mathbf{W}\mathbf{h}_u]))}{\sum_{w \in \mathcal{N}(v)} \exp(\text{LeakyReLU}(\mathbf{a}^T [\mathbf{W}\mathbf{h}_v \parallel \mathbf{W}\mathbf{h}_w]))} \quad (15.3)$$

Aggiornamento: $\mathbf{h}'_v = \sigma(\sum_{u \in \mathcal{N}(v)} \alpha_{vu} \mathbf{W}\mathbf{h}_u)$

15.6 Graph-Level Readout

Per task a livello di grafo, **gli embedding dei nodi vengono aggregati in un unico vettore di rappresentazione del grafo tramite una funzione di readout**:

- **Sum/Mean**: $\mathbf{h}_G = \sum_{v \in V} \mathbf{h}_v^{(K)}$.

- **Max**: componente per componente.
- **Hierarchical pooling**: DiffPool, MinCutPool — pool adattativi.

15.7 Espressività delle GNN

Teorema 15.1: GNN e Weisfeiler-Leman Test

Le GNN basate su sum-aggregation sono **al massimo potenti quanto il test Weisfeiler-Leman (1-WL)** per l'isomorfismo di grafi. Grafi strutturalmente diversi che 1-WL non distingue non vengono distinti da alcuna GNN con sum-aggregation. (Xu et al., 2019 — Graph Isomorphism Network, GIN)

GIN (Graph Isomorphism Network): la GNN **più espressiva** possibile con message passing standard:

$$\mathbf{h}_v^{(k)} = \text{MLP} \left((1 + \varepsilon) \mathbf{h}_v^{(k-1)} + \sum_{u \in \mathcal{N}(v)} \mathbf{h}_u^{(k-1)} \right)$$

15.8 Reservoir Computing per Grafi: GraphESN

GraphESN (Gallicchio & Micheli, 2010): estende il Reservoir Computing ai grafi. Il reservoir (pesi fissi) **elabora il grafo iterativamente fino a convergenza** (punto fisso) senza training; solo il readout è addestrato.

Vantaggi: molto **efficiente computazionalmente**; ottimo punto di partenza per analisi esplorativa su grafi.

15.9 Sfide Aperte nel Deep Learning per Grafi

- **Over-smoothing**: con troppi layer GNN, gli embedding dei nodi diventano indistinguibili.
- **Scalabilità**: grafi con milioni di nodi richiedono metodi di campionamento (GraphSAGE, ClusterGCN).
- **Eterogeneità**: grafi con tipi diversi di nodi/archi.
- **Explicability (XAI)**: spiegare le predizioni delle GNN.
- **Espressività limitata**: superare il bound 1-WL con position-aware GNN, higher-order GNN, etc.

Appendix A

Riassunto della Notazione

Questa appendice riassume la notazione usata nel corso. Si seguono le convenzioni delle slide ufficiali del Prof. Micheli, con alcune variazioni locali indicate nei capitoli specifici.

Dati

Simbolo	Significato
$\mathbf{x}^{(p)}, \mathbf{x}_p$	p -esimo pattern di input (vettore in $\mathbb{R}^n$)
$y^{(p)}, d^{(p)}, t^{(p)}$	Target/label del p -esimo esempio
ℓ	Numero di esempi nel training set
n	Dimensionalità dell'input (numero di feature)
K	Numero di classi (classificazione)
$\mathbf{X} \in \mathbb{R}^{\ell \times n}$	Matrice dei dati
$x_{p,i}$	Componente i del p -esimo pattern
$x_0 = 1$	Componente fittizia per il bias

Modello e Pesì

Simbolo	Significato
$\mathbf{w}, w_{ij}$	Vettore/matrice dei pesi (free parameters)
w_0, b	Bias
$h_{\mathbf{w}}(\mathbf{x}), h(\mathbf{x})$	Ipotesi/output del modello
$\mathcal{H}$	Spazio delle ipotesi
$f(\cdot)$	Funzione di attivazione
net_i	Net input dell'unità i : $\sum_j w_{ij}x_j$
o_i	Output dell'unità i : $f(\text{net}_i)$

Apprendimento

Simbolo	Significato
$E(\mathbf{w}), \mathcal{L}$	Loss function / funzione di errore
$E^{(p)}$	Loss sul singolo pattern p
η	Learning rate
δ_i	Segnale di errore per l'unità i (backpropagation)
Δw_{ij}	Aggiornamento del peso w_{ij}
α	Coefficiente di momentum
λ	Coefficiente di regolarizzazione (weight decay)

Valutazione

Simbolo	Significato
$E_{tr}(h)$	Errore di training (empirico)
$R(h)$	Rischio (errore di generalizzazione atteso)
$E_{val}(h)$	Errore di validation
$E_{ts}(h)$	Errore di test
$d_{VC}(\mathcal{H})$	VC dimension di $\mathcal{H}$
K	Numero di fold in K-fold cross-validation

Operazioni Matematiche

Simbolo	Significato
$\mathbf{a}^T \mathbf{b}$	Prodotto scalare
$\mathbf{a} \odot \mathbf{b}$	Prodotto di Hadamard (elemento per elemento)
$\ \mathbf{a}\ $	Norma euclidea L_2
$\ \mathbf{a}\ _1$	Norma L_1
$\mathbb{E}[X]$	Valore atteso di X
$\text{Var}[X]$	Varianza di X
$\sigma(z) = 1/(1 + e^{-z})$	Funzione sigmoide
$\text{softmax}(\mathbf{z})_k = e^{z_k} / \sum_j e^{z_j}$	Funzione softmax
$\text{sign}(z) = z/ z $	Funzione segno
$\mathbf{1}[\text{cond}]$	Funzione indicatrice

Reti Neurali

Simbolo	Significato
L	Numero totale di layer
$m, m^{(l)}$	Numero di unità nel (l-esimo) hidden layer
$\mathbf{W}^{(l)}$	Matrice di pesi del layer l
$\mathbf{h}^{(l)}, \mathbf{a}^{(l)}$	Attivazioni del layer l
$\mathbf{h}_t$	Stato nascosto al timestep t (RNN)
$\mathbf{c}_t$	Cell state (LSTM)

Appendix B

Derivazione Completa della Backpropagation

Questa appendice fornisce la derivazione formale dell'algoritmo di backpropagation per un MLP feedforward generico, seguendo le note ufficiali del corso (“Derivation of the Back-propagation based learning algorithm”, Micheli).

Setup

Consideriamo un MLP con:

- Layer di input con n unità.
- H layer hidden con funzione di attivazione f .
- Layer di output con K unità.

Loss (SSE) su tutti i pattern:

$$E = \frac{1}{2} \sum_{p=1}^{\ell} \sum_{k=1}^K (y_k^{(p)} - o_k^{(p)})^2 = \sum_p E^{(p)}$$

Il gradiente si calcola per ogni pattern p separatamente (versione online).

Case 1: Peso di Output

Sia w_{kj} il peso dalla j -esima unità hidden all' k -esima unità di output.

$$\begin{aligned} \frac{\partial E^{(p)}}{\partial w_{kj}} &= \frac{\partial E^{(p)}}{\partial o_k^{(p)}} \cdot \frac{\partial o_k^{(p)}}{\partial \text{net}_k^{(p)}} \cdot \frac{\partial \text{net}_k^{(p)}}{\partial w_{kj}} \\ &= (o_k^{(p)} - y_k^{(p)}) \cdot f'(\text{net}_k^{(p)}) \cdot a_j^{(p)} \end{aligned} \tag{B.1}$$

Definendo $\delta_k^{(p)} = -(o_k^{(p)} - y_k^{(p)}) \cdot f'(\text{net}_k^{(p)})$:

$$\frac{\partial E^{(p)}}{\partial w_{kj}} = -\delta_k^{(p)} \cdot a_j^{(p)}$$

Regola di aggiornamento:

$$\Delta w_{kj} = \eta \cdot \delta_k^{(p)} \cdot a_j^{(p)}$$

Case 2: Peso Hidden

Sia w_{ji} il peso dall' i -esima unità (input/hidden precedente) alla j -esima unità hidden.

L'errore $E^{(p)}$ dipende da w_{ji} attraverso *tutte* le unità di output (o del layer successivo) connesse a j :

$$\begin{aligned} \frac{\partial E^{(p)}}{\partial w_{ji}} &= \sum_k \frac{\partial E^{(p)}}{\partial \text{net}_k^{(p)}} \cdot \frac{\partial \text{net}_k^{(p)}}{\partial a_j^{(p)}} \cdot \frac{\partial a_j^{(p)}}{\partial \text{net}_j^{(p)}} \cdot \frac{\partial \text{net}_j^{(p)}}{\partial w_{ji}} \\ &= \sum_k \left(-\delta_k^{(p)} \right) \cdot w_{kj} \cdot f'(\text{net}_j^{(p)}) \cdot x_i^{(p)} \end{aligned} \quad (\text{B.2})$$

Definendo:

$$\delta_j^{(p)} = f'(\text{net}_j^{(p)}) \cdot \sum_k \delta_k^{(p)} \cdot w_{kj} \quad (\text{B.3})$$

otteniamo:

$$\frac{\partial E^{(p)}}{\partial w_{ji}} = -\delta_j^{(p)} \cdot x_i^{(p)}$$

Regola di aggiornamento:

$$\Delta w_{ji} = \eta \cdot \delta_j^{(p)} \cdot x_i^{(p)}$$

Generalizzazione Multistrato

Per un MLP con L layer, la formula ricorsiva si estende:

$$\delta_j^{(l)} = f'(\text{net}_j^{(l)}) \cdot \sum_k \delta_k^{(l+1)} \cdot w_{kj}^{(l+1)}$$

La propagazione parte dall'output ($l = L$) e procede all'indietro fino al primo hidden layer ($l = 1$). I gradienti fluiscono "a cascata" attraverso i pesi.

Complessità Computazionale

Per un pattern p :

- **Forward pass:** $O(d_L)$ dove d_L è il numero totale di pesi.
- **Backward pass:** $O(d_L)$ — stesso ordine del forward pass.

La backpropagation è quindi computazionalmente equivalente al forward pass — uno dei motivi del suo successo.

Note sulla Sigmoid

Per la sigmoide $f(x) = \sigma(x) = \frac{1}{1+e^{-x}}$:

$$f'(x) = \sigma(x)(1 - \sigma(x)) = o_j(1 - o_j)$$

Quindi il segnale di errore dell'output diventa:

$$\delta_k = o_k(1 - o_k)(y_k - o_k)$$

e per i layer hidden:

$$\delta_j = a_j(1 - a_j) \sum_k \delta_k w_{kj}$$